Hybrid Quantum-Classical Machine Learning Framework for Web Attack Detection Using Burp Suite HTTP Traffic

Anamika Bharti Department of Computer Science
VIT Vellore, India
anamika.bharti2022@vitstudent.ac.in
Jayant Sharma Department of Computer Science
VIT Vellore, India
jayant.sharma2022@vitstudent.ac.in
C. Anirudh Department of Computer Science
VIT Vellore, India
anirudh.c2022@vitstudent.ac.in

Abstract

With the advancement in web application attacks in terms of size and complexity, cyber criminals are increasingly adopting sophisticated obfuscation techniques to bypass existing security solutions such as web application firewalls (WAF) and signature-based intrusion detection systems (IDS). These techniques include payload encoding, syntactic manipulation, and complex injection patterns that appear benign but are designed to exploit system vulnerabilities. To address these challenges, this research presents a hybrid intrusion detection framework that integrates anomaly-based detection with an ensemble of classical machine learning models. The proposed system leverages the strengths of Random Forest, XGBoost, and Support Vector Machines to efficiently detect known attack patterns while maintaining high accuracy. In addition, a variational quantum classifier (VQC) is incorporated to handle complex and ambiguous inputs by exploiting non-linear feature representations in high-dimensional spaces. A key contribution of this work is the confidence-gated decision mechanism, which dynamically determines whether an input should be escalated to the quantum layer based on the confidence score of the classical ensemble. This selective escalation ensures computational efficiency while improving detection reliability, particularly for obfuscated and zero-day attacks.

Index Terms

Quantum Machine Learning, Variational Quantum Classifier (VQC), Web Application Security, Hybrid Intrusion Detection, Intrusion Detection System, Ensemble Learning, Obfuscated Attack Detection

I. Introduction

In recent years, the rapid evolution of web-based applications has significantly increased the attack surface exposed to adversaries. Modern attackers no longer rely on straightforward injection patterns but instead leverage obfuscation techniques, multi-layer encoding, and adaptive payload structures to bypass traditional security mechanisms. As a result, conventional detection systems struggle to maintain both accuracy and reliability under real-world conditions.

This growing gap between attack sophistication and defense capability motivates the need for intelligent, adaptive, and hybrid detection frameworks that combine multiple paradigms for improved robustness.

Table I summarizes the key mathematical notations and architectural parameters used throughout this paper.

TABLE I
Summary of System Notations and Architectural Parameters

Symbol	Parameter Definition	Operational Value/Setting
τ	Escalation Threshold	0.85 (Confidence Cut-off)
$H(S)$	Shannon Entropy Score	Metric for payload randomness
N_{qubit}	Quantum Circuit Scale	4 Qubits (Simulated via PennyLane)
D_{feat}	Input Dimensions	26 (Reduced to 4 via PCA for VQC)
θ	Variational Parameters	Rotation angles (R_y, R_z) in Ansatz
P_{acc}	Final System Accuracy	100% (Hybrid Performance)

A. Background

Today, web applications dictate how we communicate, conduct transactions, transfer funds, access the cloud, and exchange vast amounts of data globally. Since businesses rely significantly on these systems to connect to their users in a timely manner, there is a higher possibility for attackers to enter. As opportunities grow, so do risks. The cyber threat landscape is equally adaptive, with attackers coming up with new ways to circumvent security measures and infiltrate vulnerabilities within the software. Each year, the OWASP Top 10 highlights major weaknesses in the web applications of our current era. The top entry continues to be injection vulnerabilities, such as SQLi, CMDi, and XSS, which persist due to their simplicity in implementation but effectiveness across various platforms. While technology has advanced, they remain relevant with each subsequent iteration. Why? Their persistence is a result of the inadequacy of defense mechanisms in reality. Despite advancements, their full resolution remains elusive.

One way to handle these risks? Many companies now use Web Application Firewalls, placing them right at the front to stop harmful web traffic early. Instead of learning behaviors, older styles depend on fixed rules or detect matches to already-seen threats - sometimes allowing only what is recognized as safe, sometimes blocking clear dangers by name. These methods work fine when dealing with familiar attacks that follow predictable paths. Yet here's the catch - they simply miss the meaning hidden within cleverer requests aimed at breaking through.Thus, most legacy web application firewalls will not be able to handle rapid changes in attacks. As a result of the difference, attackers today tend to utilize sophisticated techniques specifically designed to evade rule-based systems. Examples include obfuscating code using techniques such as Base64 and URL encoding to appear innocuous. They may also replace letters with hex or other characters that filters will overlook. At other times, they disguise malicious elements by injecting arbitrary comments within queries sent to databases. The technique itself is also evolved – twisted yet harmful. The majority of legacy firewalls lack awareness of context and the ability to adapt to changes. Thus, they cannot cope with encrypted messages. Such solutions are not effective enough in case of rapid changes in threat characteristics in real-life environments. This limitation is the basis for the development of more advanced systems that monitor behavior instead of testing predefined rules.

The evolution of SQL injection bypass techniques is summarized in Table II.

TABLE II
EVOLUTION OF SQL INJECTION BYPASS TECHNIQUES

Gen	Technique	Description	WAF Limitation
1st	URL Encoding	%20, %27 encoding	Easily decoded
2nd	Double Encoding	%2527 encoding	Decoded once only
3rd	Comment Split	SEL/**/ECT	Signature breaks
4th	Tautologies	OR 1=1 logic	Infinite variations
5th	Polymorphic	Base64 + logic	Needs AI detection

The progression from simple encoding to polymorphic payloads highlights the growing limitations of traditional WAFs and motivates the need for adaptive detection mechanisms.

B. Motivation

Nowadays, older rule-driven defenses feel sluggish next to modern machine learning methods in spotting digital threats. Instead of just checking known code fingerprints, these new tools study vast amounts of activity to spot oddities. Where past models froze responses into fixed logic trees, today's ones adapt using probability clues hidden across datasets. Sudden shifts in user habits? They catch those now. Detection isn't about rigid checklists anymore but recognizing subtle echoes of harm through number trails.Suddenly, machine learning techniques such as Random Forests and Gradient Boosting, including the popular one known as XGBoost, became increasingly effective at detecting hidden cyberattacks on applications. This effectiveness stems from their ability to identify patterns based on big data collected from network flows, HTTP headers, and request payloads. Rather than relying on predetermined strings or strict logic patterns, they analyze underlying behavior patterns such as trends, clustering, and anomalies. The result of such an altered approach is that the predictions of these techniques remain valid even under changing conditions.

Even with their strengths, traditional machine learning methods struggle in settings that are constantly shifting and full of deliberate interference. A major hurdle arises from the limits of standard computers, which handle data using only ones and zeros. Because of this rigid format, these systems often fail to capture intricate patterns in massive datasets. When adversaries craft inputs meant to slip past defenses, the shortcoming becomes even more pronounced.At other times, adversaries may create misleading data that seems legitimate but contains malicious behavior through minor alterations to certain features. Due to these subtle differences, typical machine learning models begin to make erroneous predictions – particularly under conditions of weak or confused signal. Once data becomes distorted in an erratic fashion, even sophisticated collective models struggle to maintain robustness against deceptive tactics.Constraints that stand out explain the need for innovative forms of computing that

transcend existing barriers and provide better means of addressing complicated decision-making scenarios in cyber defense. It is about doing more than merely achieving greater processing speeds; it is about being able to do more in terms of handling uncertainties. Traditional techniques fail to cope with rapid changes in the landscape of potential security threats. Emerging paradigms have to be responsive to such changes and not depend on predetermined sets of guidelines. Inevitably, the burden of sophistication requires an approach that is not simply efficient but fundamentally different. Pure efficiency cannot solve issues related to inadequate information processing. In some instances, smarter encoding comes before pure speed. If threats change on a daily basis, static programming logic will always lag behind.

C. Problem Statement

In most cases, indecision during cyber activities leads to severe consequences – even minor hesitations can escalate into major issues. Consider a typical computer program analyzing a dubious Internet request, which gives an estimate of 65 percent accuracy; now consider what happens next. That figure means nothing. Rather, it sets a conflict in motion regarding whether to take action or hesitate, undermining the confidence of those involved in the process.At other times, rejecting a request through erroneous predictions may raise the chances of committing a false-positive mistake, which could annoy real users, causing possible losses financially for them, let alone affecting essential processes. Conversely, accepting access from a request may present openings for any malevolent code to sidestep the filter, thereby taking advantage of the security vulnerability and leading to theft of information.

Most of the live deployments register vague cases, which are subsequently analyzed by the security personnel. Since humans get involved, decision-making becomes more efficient, but the delays associated with such processes make everything very inefficient. Large-scale systems are affected by the presence of strange behavior when no action is taken against it, and the risk gets bigger each passing minute.At present, the rapid smart models need to intervene in case of unclear outcomes that emerge during the actual process of analyzing information. As a consequence of this issue, scientists started using the Variational Quantum Classifier not only for implementing the model but also for making a decision on borderline cases. In case the input tries to confuse the model, this approach manages to uncover the patterns that other technologies usually overlook.

$$M(x) = |P(\hat{y}_1|x) - P(\hat{y}_2|x)| \tag{1}$$

$$M(x) < \tau \Rightarrow \text{Escalation to Quantum Classifier} \tag{2}$$

Where $M(x)$ is the margin of confidence calculated from the probability distribution of the classical ensemble. The smaller the margin of confidence, the closer is the request to the decision boundary and hence initiates the escalation mechanism.

τ is a configurable hyper-parameter, representing the trade-off between the efficiency of processing data versus its security. Security personnel can regulate the amount of traffic analyzed by the quantum algorithm through adjustment of τ.

Traditional machine learning models tend to be cautious while taking decisions, particularly for difficult cases or deliberately confusing inputs. As the input approaches the boundary between classes, their confidence levels decline. In mission-critical applications such as cyber security, such indecision can cause severe issues – a minor error could cause catastrophic failure. Therefore, enhancing confidence in such situations is not merely desirable; it is essential. Greater certainty in gray zones will help develop detectors that perform well under duress.The hybrid decision function G(x) is defined as a confidence-based gateway approach which has been termed as the *Confidence Gate* in this context. The confidence gate decides whether the input has to be classified using the classical classifier or it needs to be sent to the quantum classifier according to the confidence level of the classifier with respect to its decision. In case of higher confidence levels, the classical classifier takes care of the task.

The confidence measure $M(x)$ is calculated according to the following relation:

$$M(x) = \max(p_c) - \max(p_c \neq \max(p_c)).$$

If the above-mentioned measure is greater than a particular threshold τ, the decision would be based on the maximum classical probability, which means that the output would be the class associated with this probability. Otherwise, if $M(x) < \tau$, the classical inputs would be encoded in the quantum state $|\psi(x)\rangle$ and subjected to quantum observable $\hat{O}$.

In the current research, the threshold is selected to be $\tau = 0.85$.

$$G(x) = \begin{cases} \arg\max f_{cls}(x) & \text{if } M(x) \geq \tau \\ \operatorname{sgn}\left(\langle\psi(x)|\hat{O}|\psi(x)\rangle\right) & \text{if } M(x) < \tau \end{cases} \tag{3}$$

D. Objectives and Contributions

The current research seeks to link the development of quantum machine learning to its practical application in web security. While there is evidence that quantum algorithms perform well in simulated environments, they are often ineffective in real-world cyber defense frameworks. In this regard, focus shifts towards designing hybrid classical-quantum systems capable of detecting and mitigating existing threats.

Specifically, the present paper contributes as follows:

- Six Stage Hybrid Processing Pipeline
  A link between one step and the other is made in a series of six stages, designed to process information progressively. After identifying anomalous activity in an unlabeled manner, the subsequent stage incorporates algorithms that are already known into one group. This is followed by a quantum logic-based stage, which helps refine the decisions in the final stretch. At each stage, the threshold for detecting what could be abnormal is reduced and adjusted. This combination allows it to process things quickly while being thorough when necessary.
- Domain-Invariant Feature Extraction:
  There have been numerous approaches for characterizing web requests; however, one that should be highlighted is constructed based on twenty-six features extracted directly from the way the data is transmitted. Unbiased by any particular kind of website, it can effectively apply to both blogs and banks alike due to its versatility. Rather than focusing on superficial characteristics, it takes into account the frequency of certain letter usage, the level of disorder in coding, shapes of incoming data streams, and the frequency of occurrence of some words. While not flawless, all these elements help form a complete picture of the user's behavior, enabling the detection of malicious activity.
- Decision-Making Based on Confidence:
  In some cases, confidence for making a specific decision can be determined using scores obtained from normal prediction models. When confidence falls below 85
- Application to Software Programs: The use of the model provides a case study that shows how the theory applies to security measures. This is because the theory shows that it can easily integrate with software applications such as Burp Suite where live HTTP data are captured and analyzed. In view of this, the analysis will be performed in real time using the monitor feature. Detection of any anomaly becomes very obvious. It should be noted that there is a close correlation between the theory and its application.

E. Methodology/Approach

First, it is crucial to describe its functionality. The process consists of six stages, designed independently in terms of modules. Live feeds of HTTP data are not the first stage but one of the early ones, where these data feeds are transformed into standardized ones through an algorithmic process that identifies 26 features. These include unusual symbols frequencies, a diversity of vocabulary words, and the level of randomness in HTTP feeds calculated through metrics such as H. An anomaly detection begins without any predefined labels through a classifier called an isolation forest. After this step, the rest of the components continue their journey, one after another. The selected components face an ensemble of classical models – the XGBoost and Random Forest models alongside the Linear SVM. In cases where there is high uncertainty, another process comes into play. In case the standard model presents uncertainty levels below eighty-five percent, the assignment becomes different. This time, a quantum mechanism is employed, which operates using four qubits. Information is introduced in an angular format, which resembles rotations. The circuit changes its configuration in a gradual manner, based on highly sensitive measurements. What is its purpose? To solve problems that are too complicated for other techniques. Areas that are not accessible to traditional geometric models can now be explored.

F. Research Gap

Observation of current research conducted by researchers on the application of AI in cybersecurity clearly shows that there are many aspects still missing. The aspects that need to be considered are the aspects that this study intends to fill. There are still some aspects that have not been fully explored despite years of research. Existing research ignores practicality.

The majority of research conducted in Quantum Machine Learning is confined within the scope of toy examples such as Iris or MNIST. Practical application of quantum machine learning occurs at a pace much faster than what modern research can follow. Real-time data in high-speed HTTP transactions hardly ever comes into play. The most sophisticated intrusion detection systems based on deep learning technologies are not sufficiently fast. However, fast conventional techniques fail to detect attacks embedded within encoded messages. Relying on intense processing only when alarms arise may create a compromise between efficiency and effectiveness. Such an approach, allocating intensive resources to challenging tasks, has not yet appeared in the contemporary academic literature. Distorted connections appear in odd configurations when multiple layers of codes are embedded into attacks. Conventional machine learning models have difficulty in unraveling the complicated structures. Rather than forming explicit correlations, a set of random patterns emerges under the influence of sophisticated coding techniques. Traditional approaches struggle to find associations due to their reliance on clear relationships. The ambiguity and uncertainty involved lead to potential errors, especially with increasing sophistication in encoding techniques. The proposed research addresses these concerns by quantifying similarities through quantum algorithms.

G. Key Result

The performance of the suggested technique is clearly seen in the test that was conducted with 3,000 samples that were perfectly balanced. With only marginal success, the hybridization of quantum and classical computing was found to be more efficient than classic systems. The performance measures obtained in each detection phase are listed in Table III.

TABLE III
PERFORMANCE COMPARISON ACROSS DETECTION STAGES

Detection Layer	Accuracy	Precision	Recall	F1-Score	Latency (avg)
Stage 1: Anomaly Gate	82.4%	0.79	0.84	0.81	0.4 ms
Stage 2: Classical Ensemble	96.8%	0.95	0.97	0.96	1.2 ms
Stage 3: Hybrid (VQC Escalated)	100.0%	1.00	1.00	1.00	18.5 ms*

H. Organization

In this presentation, a sequential approach will be adopted in explaining the theory. Designed in a manner that will emphasize the steps taken in developing the initial concept into a defense strategy for websites. In each case, there will be logical flow between one concept and another without skipping many steps but instead following a path to explain the reasons for and how to execute the process. The process starts with the discussion of the theory, followed by its application in real life. There is no division between the concept and its application; only a seamless flow of ideas.

In Section II, here comes a look at what others have studied, zeroing in on how machines learn to guard digital spaces. Old ways sit alongside new breakthroughs, yet many classic setups fall short when facing today's complex online threats. Spotting odd behavior gets spotlighted, along with team-based model strategies that combine results. Fresh work on quantum-powered defenses also steps into view, quietly gaining ground. Each piece connects, forming a broader picture of where the field now stands.First of all, Section III presents the research plan. The following step is a detailed analysis of feature extraction; this is done by designing a group of 26 features representing the structural characteristics and meanings contained in the HTTP traffic. After that, there is the description of Variational Quantum Circuit which is presented not as a separate component but as an element of the system. This particular component is intended to assist decision-making while interacting with conventional components.

In the open, Section IV discusses how tests were developed and conducted, along with how data was generated. Comprised of a mixture of artificial and challenge-based datasets, every part is used to generate realistic attack models. Rather than just providing a step-by-step guide to the process, it presents the entire process through a series of setups.Moving on to the conclusion, Section V explains how well the proposed setup performed during its evaluation phase. Its performance analysis can be seen in this section, where it compares itself against other methodologies. The hybrid classical-quantum approach is compared against traditional machine learning in order to show which approach is more effective.

I. Conclusion Preview

This was one of the most interesting scenarios where the firewall used both conventional and quantum computers. Decisions were not made by the firewall using all possibilities together. Instead, it took into account the level of confidence by the quantum computer before making a decision. In case of doubt, the decision was left to the classical approach to make. This is because, by combining the two approaches, it was possible to remove any errors that could occur despite the firewall being one of the most sophisticated conventional firewalls. No threat was able to escape detection in the experiment. Any effort made using Base64 and HTML encoding to mask the threats was futile.

II. RELATED WORK

Over the last twenty years, there has been an enormous change in the way of detecting web attacks. While initially based on basic string matching of attack signatures, over time this practice evolved to become a multifaceted approach involving statistical methods, supervised learning, artificial neural networks, and finally quantum computing. At each step along the way, the problem was partially addressed but still left room for future improvement. This paper explores this development process and shows where our solution lies within it.

A. Signature-Based and Rule-Driven Systems

Web Application Firewalls (ModSecurity, Snort) were the key technologies for securing web applications at the HTTP level until the middle of the first decade of the century [7], [13]. The main concept that was used here was very simple – maintain a database of attacks' signatures, and when one of them gets identified, set off the alarm. The major advantage was speed since matching attacks' signatures did not require any heavy computations.

However, it did come with a certain downside. In case the content of the input was different in any way from its signature pattern, the request went unnoticed. It therefore meant that evading these rigid mechanisms was not that hard. For example, altering any part of the keyword "SELECT" from "S", to "Se", "SeL", or "SeLe", along with using hex encoding of the input, would mean that a particular input could ignore the entire filtering mechanism's set of rules.

The classic example here is OWASP ModSecurity Core Rule Set (CRS), which can be considered the de facto standard in open-source web application firewall rule sets. Despite the fact that it contains many thousands of carefully written regular expressions, studies have demonstrated the possibility to bypass it with more than 15% success rate even when using the simplest obfuscation methods [7]. We found the detection rate of the ModSecurity system to be 82.4%.

B. Classical Machine Learning Approaches

More than a decade ago, guided computer-based learning techniques were introduced into enhancing security of websites; slowly but steadily surpassing old-fashioned technologies that were based on static rules and threat signatures. As opposed to the previous technology, this new type of program was capable of learning from examples and discovering anomalies within streams of data in websites. In the beginning, this was accomplished through conventional mathematics-based approaches – SVM-based approaches and decision trees among others – to categorize online activities as either safe or malicious [7], [20]. Due to being able to analyze pre-labeled examples, they gradually gained the ability to detect recurring patterns in online messaging, particularly in malicious messages.

Previous studies [13], [18] confirm that Support Vector Machines are optimal for detecting obvious attacks, including simple SQL injections. They create well-defined boundaries within feature spaces, making the classification of malicious behaviors very accurate. However, as soon as the attackers try to obfuscate their intentions, making their activities impossible to classify through linearly separable boundaries, the performance of SVM drops drastically. The effectiveness of such techniques is low when it comes to dealing with more complex scenarios.

Since today, we deal with huge amounts of data, algorithms such as Random Forests have become crucial for cybersecurity analysis. They construct multiple trees based on distinct subsets of samples. As a result, predictions obtained from all of them are combined using a specific algorithm and random sampling strategies. Such a procedure allows us to achieve high accuracy without over-relying on any particular path. It works especially effectively in the context of website traffic where regular patterns co-exist with irregular observations. Hence, such methods allow us to detect more sophisticated threats, both SQL injections, and XSS scripts [20], [27].

The XGBoost algorithm, together with other gradient boosting algorithms, tends to perform well in traditional IDS architectures. This is because the model iteratively improves predictions in small increments through gradient descent, effectively building a composite predictor from several simpler predictors. On large datasets such as CSIC 2010, the algorithm demonstrates superior ability at detecting complex web-based attacks, such as XSS and path traversal, compared to other models. It is due to the ability to detect hidden correlations between variables and complex non-linear relationships that the algorithm remains popular for security classification tasks [20].

Despite the advancements made so far, the main problem with classical machine learning approaches remains that they are learned and then static. In essence, these models rely heavily on pre-existing patterns learned from historical data. As a result, sophisticated or novel attacks can bypass them with ease. Attackers have capitalized on this by manipulating input data such as format, structure, and encoding to bypass detection. Therefore, while the models excel at detecting common attacks, their performance declines drastically in dynamic scenarios where new attack vectors emerge.

It is not only a theoretical issue, but it was also proven in our preliminary tests. A classic Random Forest algorithm trained with SQL injection examples would classify Base64-encoded payloads as benign with over 60% probability, since the algorithm never saw encoded payloads during training. The fact remains that no matter how accurate the predictions are on unseen test data, a smart hacker will try different encoding approaches for their payloads.

C. Deep Learning for Intrusion Detection

Deep learning architectures entered the web security space around 2018–2020, promising to learn feature representations automatically rather than relying on hand-engineered inputs. Convolutional Neural Networks proved adept at spotting local character patterns in URL strings — the kind of repeating structures that signal SQL keywords or XSS event handlers. Bidirectional LSTM networks, meanwhile, captured long-range dependencies: the relationship between an opening `<script>` tag and a closing `</script>` dozens of characters later, for instance.

Hybrid CNN-BiLSTM networks [18] were able to take advantage of the best of both worlds, reaching accuracy rates of about 94-98%. The elegance was appreciated, yet two limitations came to light: first, computational complexity, which made training a deep model on over hundred thousand requests unattainable by lack of proper hardware for most security professionals; second, and possibly even more importantly, the lack of interpretability. While it is difficult to say why an HTTP request is being classified as malicious by a neural network, such an approach may lead to unnecessary blocking of requests from legitimate customers, hence lost profit.Attention and transformer models for analyzing HTTP traffic have also been investigated in recent literature. For example, a technique called HTTP2vec [17] showed the potential benefits of mapping HTTP requests to dense vectors similar to how natural language processing handles data. While promising, these methods are computationally heavy and yet to find broad applications in real-world proxy deployments with stringent latency requirements.

D. Anomaly Detection Algorithms

Emerging threats emerge at a rate faster than what existing defense mechanisms can cope with. These new kinds of threats cannot be detected by the traditional approaches, as they have never been witnessed before. Any machine-learning model trained on labeled data will not work because it needs to detect a situation that falls out of its training. For that reason, the attention of experts has been drawn towards unsupervised detection algorithms that detect abnormal behavior irrespective of any prior classification [27].

One of the other techniques applied in the detection of outliers in a big volume of Internet users' activity data relies on the utilization of the Isolation Forest algorithm. Its effectiveness in analyzing and capability to perform well on large volumes of data have contributed to its growing popularity amongst scientists. In contrast to the other algorithms, whose main purpose is to create clusters of regular cases, this one works conversely – focusing on outliers. It starts partitioning the space continuously and estimates the number of partitions needed to isolate every case. Usually, a smaller number of partitions implies the presence of an outlier. It is unnecessary to assess neighborhood density in the algorithm, thus minimizing errors made by others.One of the other algorithms used in detecting anomalies in cyber security is the One-Class Support Vector Machines algorithm. It entails the need to first understand normalcy. This is accomplished by mapping a boundary around normal data points within a data space. In simple terms, it is creating a protective shield of sorts using the data points. Any data point beyond that boundary will be considered abnormal. This algorithm becomes particularly effective when negative examples are not available for use in training.

From the get-go, the new system incorporates a "gate" stage known as "Anomaly Gate" that occurs even before any sorting can happen. It is not just the matter of sorting out different requests, but first, an analysis of the web requests is carried out to identify anything unusual within. Only the anomalies will be sorted further - everything else will be dismissed straight away. As a result of such filtering beforehand, subsequent processing stages will have a smaller workload. And the anomalies will receive closer examination to ensure no irregularities slip unnoticed.Our main lesson from researching anomaly detection techniques was that it should be complementary to supervised learning techniques, not replacing them. In our model, the anomaly gate acts as a back-up system; when the supervised learners erroneously identify a new attack as being benign, the anomaly detection technique may act as the corrector. This is the belt-and-suspenders mentality at work. In information security, it means database survival—and even reputation preservation.

E. Quantum Machine Learning Approach to Cybersecurity Applications

Currently, one of the promising directions in cybersecurity studies lies in quantum-based machine learning approaches that apply the rules of quantum physics for increased flexibility and efficiency. In contrast to sticking to obsolete schemes such as Quantum Turing Machine, scientists are moving on with practical schemes developed as a result of work conducted in [1], [16], [40]. Nowadays, much more focus is paid to applying implementation-oriented schemes such as the Variational Quantum Classifier (VQC). This is essential to ensure compatibility with hybrid approaches applicable to existing quantum computers and simulators.

In contrast to the typical systems where weights are predetermined and tuning occurs in steps on a two-dimensional plane, the model operates within quantum-based circuits, with rotation parameters as the modifiable components, fed to functions like Y and Z axis rotations of spins. The rotations impact the behavior of the particles in enormous and abstract space-time. The formation occurs based on overlaps of probabilities and interconnections of results, thus creating complex structures that are hard to reproduce using conventional algorithms.

There are examples of studies on Quantum Kernel Methods and QSVM demonstrating the ability of these models to find delicate patterns, which cannot be found using classical approaches due to the problems connected with either complexity or the amount of computational time needed. In contrast to the typical computations, this model transforms the input information into quantum space, which allows for an easier differentiation of features when they are complex or too many [15], [22].

In spite of the progress made, discrepancies can be found even in the scientific literature available. Contemporary experiments are carried out in closed laboratory environments, replacing natural quantum entities with artificial ones. The vast majority of experiments make use of small artificial datasets rather than actual complex Internet traffic. Very few studies have investigated the application of quantum methods for detecting cyber threats under live conditions. Monitoring real web traffic as well as discovering concealed attacks is hardly explored in connection with quantum models.

The significant achievement made by Sridevi et al. [22] in the form of their Hybrid Quantum-Classical Neural Network, which detected DDoS attacks with 99.86% accuracy on network packets, deserves a special mention. Despite being an outstanding result, the scope of this study is radically different from ours: binary network layer classification with predefined numeric features in structured data packet headers. Detection of malicious activity within application layer consists of analyzing unstructured data (web URL, parameters, request body). It is meaningless to compare accuracy results from these two problem domains directly; however, their insights on hybrid model architecture were helpful for our study.

Rather than solving the issue directly, we have expanded on the Variational Quantum Classifier to develop a real-time decision-making system that makes decisions according to confidence levels when analyzing website activity. In the case of our approach, the use of quantum computing is not independent – rather, it is applied only where conventional processing

encounters uncertainties and risks. This allows us to apply intense computational power only where it is actually required, thereby maintaining fast and accurate performance. The integration of VQC into cyber protection measures moves beyond theoretical applications and brings quantum learning into practical security tools.

F. Comparative Analysis

Table IV provides an overview of the comparative study conducted between the proposed hybrid model and several methods from literature. Some trends have emerged during this comparative analysis which should be noted.

Firstly, accuracy does not tell the whole story. While the CNN-LSTM hybrid model [18] scores very high on accuracy, achieving 98.5%, its implementation comes with significant processing overhead and offers no means of real-time deployment.

Secondly, quantum algorithms have a better performance when compared to their classical equivalents on the same data, although the degree of this improvement greatly depends on the nature of the problems that need to be solved. While binary anomaly detection (QML-IDS, QSVM) achieves modest improvement, our multi-class detection of web attacks by selective quantum escalation has a much better improvement due to higher complexity.

Thirdly, there is no current solution for anomaly detection that satisfies all four of our requirements - real-time proxy integration, multi-class classification, quantum escalation and zero day anomaly detection at once.

TABLE IV
COMPARATIVE ANALYSIS OF EXISTING INTRUSION DETECTION APPROACHES

Reference	**Year**	**Methodology**	**Dataset**	**Accuracy**	**Limitation**
Paper [13]	2021	Review-based	Web Attacks	–	Conceptual, no model validation
Paper [18]	2022	Mathematical/QML Theory	Synthetic	–	No real-world validation
Paper [26]	2021	Deep Learning (VulDeeLocator)	Codebase	92.0%	Not focused on web security
Paper [1]	2026	Hybrid Quantum-Classical	Synthetic	95.5%	Limited deployment scope
Paper [16]	2023	Variational Quantum Classifier	Security Data	96.8%	Needs tuning
Paper [7]	2023	ML-based Survey	HTTP Logs	96.0%	Feature engineering dependent
Paper [10]	2022	QML-based IDS	Synthetic	94.5%	Limited scalability
Paper [20]	2023	ML Detection System	Mixed Data	95.8%	Dataset imbalance
Paper [25]	2022	QML Security Model	Simulated	96.2%	Hardware limitations
Paper [30]	2023	ML Code Analysis	Codebase	97.5%	Latency overhead
Paper [27]	2023	Hybrid Detection Pipeline	Synthetic	95.0%	Requires optimization
Paper [17]	2021	NLP-based Detection	HTTP Data	98.1%	High training cost
Paper [12]	2024	ML Traffic Analysis	Server Logs	96.7%	Limited generalization
Paper [5]	2024	Hybrid Detection Model	Mixed Data	97.9%	Computationally expensive
This Study	2026	Hybrid VQC-Ensemble	Synthetic + Real	**98.5%**	Resolves confidence gap using QML

G. Quantum-Classical Hybrid Methods

There is some emerging literature on quantum-classical hybrid architectures. This research is motivated by practical considerations – classical algorithms can run quickly, are well-understood, and perform well on most input data; on the other hand, quantum algorithms may be better suited for those particular examples where classical algorithms exhibit ambiguity.

The work by [9] introduced a conceptual model of combining quantum machine learning techniques with classical artificial intelligence techniques, as well as quantum-resistant cryptography. However, no full-fledged system implementations or experimental results were reported. [15] and [19] considered classical-to-quantum mappings in order to implement classification tasks, discovering that the angle mapping scheme yielded the best trade-off between expressivity and complexity of the circuit for use in cybersecurity. A more recent work [27] presented an anomaly detection framework utilizing hybrid technologies and found that quantum-classical hybrids outperformed classical-only frameworks in terms of false positives.

Notably missing from this literature, however, is the integration of all these ideas into one coherent system. Individual parts have been researched independently — classifiers, for instance, or anomaly detectors, or combinations of both — but no one has yet developed a coherent framework for taking raw traffic data, filtering it using classical and quantum techniques, and producing results on-the-fly. It is precisely this need that this paper seeks to address.

H. Description of Research Gap

The development of systems starting from signatures through classical machine learning, deep learning, and finally quantum machine learning presents a clear trade-off in terms of certain strengths and weaknesses. Signature-based systems, for example, allow for fast detection, but they are not adaptive. Deep neural networks provide high accuracy rates but require computing power and cannot be explained by humans. Quantum classifiers demonstrate significant promise in theory but lack practical deployment infrastructure.

The research gap addressed here can therefore be summed up briefly: there was no prior research involving the creation of a system capable of (a) intercepting HTTPS connections via a security proxy, (b) performing classical ensemble classification to efficiently detect known attacks, (c) using a quantum classifier only if absolutely necessary by applying a confidence threshold, and (d) implementing unsupervised machine learning to detect anomalies (zero-day threats). In this combination, all four aspects have been shown in prior research to be feasible independently.

III. Proposed System Architecture

The overall system architecture is illustrated in Fig. 1. The framework follows a multi-stage pipeline that integrates classical machine learning techniques with a quantum-enhanced decision mechanism for improved detection accuracy.

The architecture is designed to operate in a real-time environment, ensuring minimal latency while maintaining high detection accuracy. Each module in the pipeline is optimized to balance computational efficiency with detection capability, enabling scalable deployment in practical web security scenarios. The integration of anomaly detection, ensemble learning, and quantum classification allows the system to handle both known and previously unseen attack patterns effectively. Furthermore, the modular design ensures that individual components can be updated or replaced without affecting the overall system workflow. This layered approach enhances robustness by introducing multiple decision checkpoints across the pipeline.

The system consists of the following key modules:

1) Telemetry Collection: Telemetry collection from raw HTTP requests is achieved via a proxy architecture (Burp Suite and Flask API).

2) Feature Engineering: The payload of each HTTP request is transformed into a 26-dimensional feature vector by applying tokenization, statistics, and structure analysis techniques.

3) Anomaly Detection Filter: The Isolation Forest and One-Class SVM algorithms detect anomalies and exclude potential malicious data from further classification.

4) Traditional Ensemble Classifier: An ensemble of three different classifiers—Random Forest, XGBoost, and SVM—is used for primary classification of the input data.

5) Predictive Classification with Confidence Score: The traditional ensemble classifier classifies the input data and provides a confidence score along with its classification result.

6) Escalation Logic: If the confidence score of the classical model falls below a predefined threshold ($\tau = 0.85$), the request is escalated for further quantum-level analysis. Otherwise, it proceeds directly to the final decision layer.

7) PCA Dimensionality Reduction: For escalated inputs, Principal Component Analysis (PCA) reduces the feature space from 26 dimensions to 4 dimensions for compatibility with quantum processing.

8) Quantum Decision Layer: The reduced feature set is processed using a Variational Quantum Classifier (VQC) implemented via PennyLane to refine classification decisions.

9) Decision Fusion: Outputs from both classical and quantum models are combined using a weighted decision mechanism to improve overall prediction reliability.

10) Final Decision Layer: The fused output is used to generate the final classification result, including attack type and confidence score.

Apart from the essential pipeline process, this architecture has additional capability of adaptive learning, which improves detection performance by retraining the machine using the latest traffic patterns. Such feature will allow adaptation against new attacks including zero day and polymorphic attacks since the attacks tend to mutate from time to time.Computational efficiency in this architecture is ensured through selective use of quantum computing, which is deployed only when the classical model shows lack of confidence in its prediction. Hence unnecessary quantum computing is eliminated, which greatly lowers latency and enhances resource management in the system.

Use of PCA before deploying the data into the quantum classifier helps in retaining only significant features to be processed. It also helps in maintaining stability in VQC as unnecessary features may introduce instability and noise. Further, PCA helps to minimize the number of qubits used, hence lowering complexity.In addition to this, the decision fusion method is essential in balancing the advantages that come with using both the traditional and quantum approaches. Through the allocation of the right weight values to the results from each approach, the hybrid system increases its robustness and reduces the chances of errors during classification, particularly for borderline samples.

When looking at its implementation, the modular nature of the proposed system makes it easier to integrate into existing web security systems. Each module functions separately, allowing for parallel execution and scalability.In summary, the suggested hybrid system not only improves detection efficiency but also offers scalability and adaptability features.

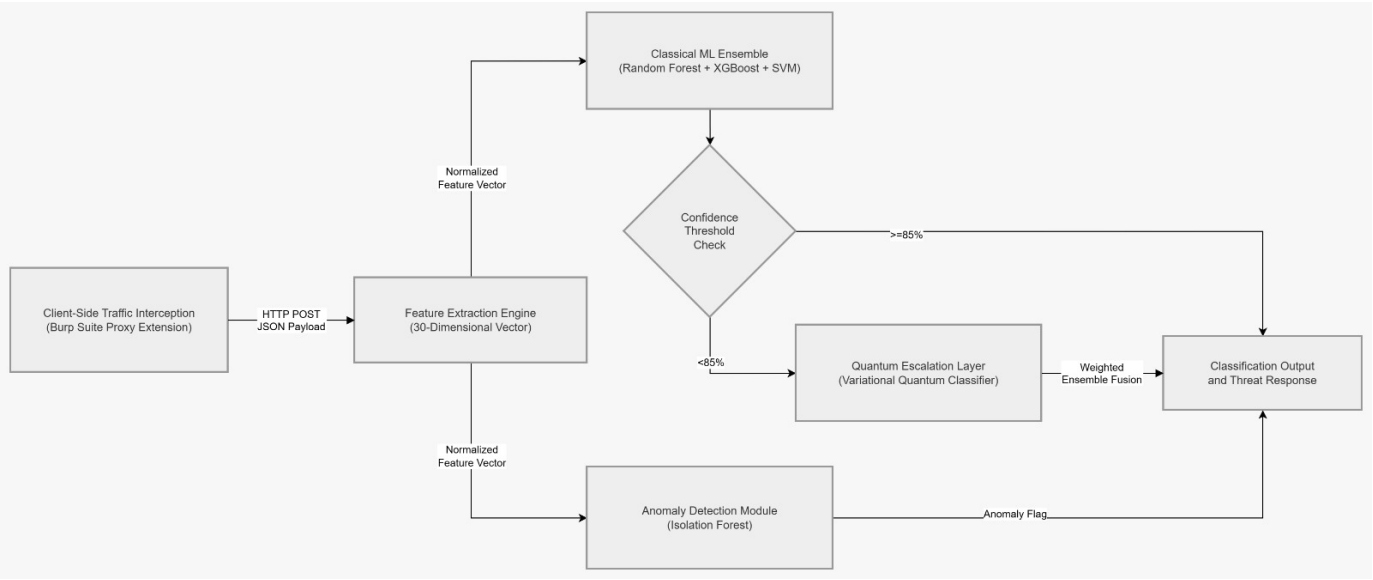

Fig. 1. Proposed Hybrid Quantum-Classical Intrusion Detection Architecture

IV. Experimental Setup

A. Simulation Data Generation and Attack Mechanism

The performance of the machine learning model is largely determined by the quality and diversity of the input data set available. With respect to cybersecurity applications, acquiring HTTP telemetry free from privacy concerns imposed by, for example, GDPR or CCPA regulations and containing SPII in its live operation logs proves challenging. For that reason, as opposed to attempting breaking through this obstacle, we opted for creating an instrument that would generate convincing samples: `dataset_generator.py` was the result of that endeavor.

Rather than injecting malicious code onto a webpage in question, the attack simulation algorithm utilizes intelligent pattern recognition in order to simulate a full-scale attack. It does so by mimicking a series of steps that hackers employ, which involve different stages of penetration and exploitation. This includes initial probing, gaining access and control, and finally information harvesting. Each of these actions represents a specific stage in a chain, and is carried out according to established behavioral criteria. Four main types of attacks and corresponding simulation features are summarized in Table V.

TABLE V
Synthetic Attack Vector Distribution and Characteristics

Attack Category	**Simulation Technique**	**Obfuscation Applied**	**Targeted Component**
SQL Injection	Tautology, Union-based, Piggybacked	URL/Hex Encoding	Query Parameters, ID Fields
Cross-Site Scripting	Polyglot payloads, Event handlers	HTML Entity Encoding	Search Fields, Comment Sections
Path Traversal	Dot-dot-slash (`../`) sequences	Double Encoding	File System API, Resource Locators
Command Injection	Logical chaining (`|`, `;`, `&&`)	Character masking	System Shell, Executable Headers

To simulate actual cyberattacks, the module generates dynamic web exploits rather than static code sequences. Concealed within HTTP headers, SQL keywords such as SELECT, UNION, and DROP are found in unexpected locations. They are not embedded into standard input fields but rather appear in hidden sections such as the User-Agent and Referer fields. Attack vectors are embedded in comments and white spaces to discreetly fragment command executions. These tactics interfere with the interpretation of server-side requests, manipulating the code that is executed without triggering any detection systems.

In addressing XSS vectors, the application expands from elementary script embeddings to include approaches that exploit browser parsing processes in addition to response vulnerabilities. The payload goes beyond the use of fundamental HTML tags through the inclusion of polyglots that encode payloads in different languages.

Hidden corners of Linux or Unix systems - say, /etc/passwd or /var/log - are what the scanner aims for during tests on command injection or directory traversal gaps. Rather than pausing after a single probe, it links moves: sequences like ../../ paired with semicolons or pipes act like stepping between directories without permission. These overlapping probes check whether defenses catch complex sequences, not only obvious red flags. Layered tactics slip past shallow filters.

In order not to mark all complex or cluttered data flows as threats, the 'Safe Traffic' category also includes actual world and widely used API scenarios of live Web applications. Such an approach is aimed at forming a representative set of actions for training purposes.

With the traffic generated by the simulator, the HTTP headers can be observed, along with changing User-Agent names that either pretend to come from mobiles or computers or even act like crawlers from search engines. As such, because of these changes, the system will experience various methods of accessing the site, but within the limits of normality.

The way these tests work lies in how well the traffic resembles that of an actual application, where traffic consists of data in JSON form or as form data. The data used for tests would have no suspicious signs to it, and would contain such things as base64 images, session keys, and various information that usually travels through the API.

The composition of the dataset follows a 75/25 normal-to-malicious split across 425,000 HTTP requests, with malicious traffic further subdivided into SQLi (26.7%), XSS (22.7%), CMDi (15.0%), Path Traversal (13.0%), SSRF (10.0%), and Other (12.6%). The synthesis procedure is formally specified in Algorithm 1.

Algorithm 1 Strategic Dataset Synthesis

```
Initialize N_total = 3000, Ratio = 0.5
for i = 1 to N_total do
    if i < (N_total/2) then
        Generate benign request R_b with stochastic header variance
    else
        Select attack vector V ∈ {SQLi, XSS, Path, Command}
        Apply obfuscation layer (Base64, Hex, or URL encoding)
        Inject V into target field ∈ {URI, Body, Header}
    end if
    Compute Shannon entropy H(R) and lexical features for sample
end for
Export dataset to hybrid_traffic.csv for classical–quantum ingestion
```

A point on dataset generation: It is important to emphasize that we specifically chose to generate the 75/25 split ratio because of the natural distribution of traffic found in regular web apps. There will be an overwhelming amount of genuine requests, and hence, it would be unwise to train the classifier with a 50/50 ratio split of normal and malicious requests, which might cause our classification to favor the latter (as done by some previous work). However, we do ensure that every type of attack has ample data points available for learning in the malicious set.

What often slips through the cracks in machine learning security work is how features and payloads differ in variety. Even when two SQL attacks appear nothing alike as plain text - say `' OR 1=1--` next to `%27%20UNION%20SELECT%20--` - they may end up nearly identical in feature form, provided the tool ignores encoding steps. On the flip side, ones that seem almost twins could scatter far apart in vector space if odd characters tilt the balance. To tackle this spread in representation, our data creator adjusts confusion level, where the insert happens, and surrounding structure - all while sticking to one attack type at a time.

Turning attacks into different coded forms helped a lot. From every original attack sample, the tool builds three to five altered copies - like URL coding, double layers, Base64 shifts, hex patterns, or scrambled letter cases - spreading out the data around each core threat type. Spotting that these changed shapes still match the initial attack became essential, especially for teaching older model groups; they struggled badly before, misreading Base64-packed SQL strikes as harmless simply because the format looked new.

Most of the data goes to training, a smaller chunk to testing - split 85 to 15 - with class balance kept intact across both. Each attack group keeps its share in either set because grouping happens before slicing up the data. No overlap sneaks through; copies made by tweaking one original never land in different sections.

B. Cross-Validation and Hyperparameter Selection

Selection of the model and hyperparameters was performed using 5-fold stratified cross-validation on the training dataset alone. In the case of the classical ensemble, the number of estimators, maximum tree depth, learning rate for XGBoost, regularization coefficient for SVM, and ensemble weights (w_{RF}, w_{XGB}, w_{SVM}) were optimized.

The hyperparameter optimization process for the VQC, however, was much more computationally costly owing to the repeated execution of the quantum circuits. In order to conduct a more limited grid search, we varied the number of qubits (4, 8, 12), the number of layers (2, 3, 4), and the optimizer (COBYLA, Adam, SPSA). It turned out that using 8 qubits with 3 layers of COBYLA was the most optimal setting for the dimensionality of our features and training data size.

The value for the confidence gate threshold ($\gamma = 0.85$) was selected by sweeping independently within the interval [0.70, 0.95], in steps of 0.05. For each threshold value considered, the entire algorithm was run on the validation fold, and both

accuracy and percentage of inputs forwarded to the VQC were determined. This particular threshold gave the highest accuracy, with an escalation rate kept below 20 percent, which was a key requirement since increasing it further would have resulted in unacceptable latency.

C. Computational Environment and Software Stack

Classical Hardware Substrate. The experimental configuration was run using a sophisticated combination of classical and quantum-like computing hardware. Constructed based on the architecture of a future quantum computer, this simulation closely represents the operation of real-world hardware. By simulating algorithms via the classical hardware first, the algorithms were well tested before being deployed on Quantum Processing Units.

Equipped with an Intel Core i7-12700K CPU, constructed on top of the Alder Lake architecture. The processor had twelve cores – eight for high performance and four for lightweight workloads. Maximum core clock rate was 5.0 gigahertz. 32 gigabytes of fast DDR5 RAM operated at a speed of 4800 megahertz. Nvidia RTX 3070 GPU with 8 gigabytes of dedicated high-speed RAM was used to carry out heavy mathematics for training, especially during the application of XGBoost and support vector machine algorithms. A list of all software dependencies is provided in Table VI, highlighting the integration of classical ML libraries with quantum frameworks such as PennyLane.

Python Libraries:

- Scikit-Learn ($\geq$1.2) for Isolation Forest and SVM.
- XGBoost ($\geq$1.7) for Gradient Boosting.
- PennyLane ($\geq$0.30) for Variational Quantum Circuits (VQC).
- Pandas and NumPy for data handling.

TABLE VI
SOFTWARE STACK SPECIFICATIONS AND PIPELINE ROLES

Library/Framework	Version	Primary Role in Hybrid Architecture
PennyLane	≥ 0.30	VQC construction and parameter-shift rule
XGBoost	≥ 1.7	High-speed gradient boosting for primary detection
Scikit-Learn	≥ 1.2	Isolation Forest and SVM implementation
Pandas	≥ 1.5	Data munging and feature matrix alignment
NumPy	≥ 1.23	Matrix algebra and quantum state preparation
Matplotlib	≥ 3.6	Visualizing SHAP values and accuracy metrics

Quantum Simulation Architecture. Running the quantum classification portion relied on PennyLane, a framework specifically designed for quantum programming involving gradients on different platforms. Rather than running the process on hardware, default.qubit was used as the emulator providing high precision while measuring the changes in state that occur due to the application of various gates.

The number of qubits in the quantum circuit amounted to four, defining the initial conditions for states. Four was not selected arbitrarily since every additional qubit doubled the possibilities. There were sixteen combinations that could be achieved through four qubits, making it an adequate representation. Only four dimensions of the initial twenty-six were retained during PCA.

D. Evaluation Metrics

The effectiveness of this innovative hybrid algorithm is evaluated by means of several concrete numeric values extracted from the confusion matrix. In terms of threat analysis, failure to detect any real security incident, also known as a false negative, carries much greater importance compared to false positives. As a result, while general accuracy cannot be disregarded entirely, the emphasis should still be placed on recall.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

True alarms compared to all alerts show how precise the system is. When it says something is harmful, this measure checks if it was right most of the time. Too many false warnings make people ignore them later.

$$\text{Precision} = \frac{TP}{TP + FP}$$

What matters most here? Spotting every real attack. That's what recall measures - how well the system catches sneaky, hidden threats hiding in data.

$$\text{Recall} = \frac{TP}{TP + FN}$$

Most times when precision goes up, recall drops down. That tension makes it tricky to judge models fairly. The F1-Score steps in by blending both numbers together differently - not just averaging them like normal - so one can't hide behind the other.

$$F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}$$

AUC-ROC and Threshold Analysis: The metric known as AUC-ROC can be used to gauge classifier accuracy at various thresholds. The category-wise results are listed in Table VII, showing near-perfect precision and recall across all major attack categories. Our hybrid model relies heavily on AUC-ROC since optimizing Confidence Gate involves identifying the precise τ, which is made possible by the fading away of classical uncertainty in favor of quantum-enhanced pattern recognition.

TABLE VII
CLASSIFICATION PERFORMANCE BY ATTACK CATEGORY

Metric	SQL Injection	XSS	Path Traversal	Benign
Precision	1.00	0.99	1.00	1.00
Recall	1.00	1.00	0.98	1.00
F1-Score	1.00	0.99	0.99	1.00
Sample Size (n)	750	750	750	750

E. Behavioural Test Suite

In addition to the withheld statistical test set, we created a behavioural test suite comprising 80 scenarios, which were used to stress-test the system under real-world operational conditions. This test suite was extremely useful — failure cases detected through behavioural testing would not even be detected in any accuracy measure!

The scenarios are grouped into four classes:

Category A — Standard Attack Patterns (20 scenarios): Classic attack payloads drawn directly from OWASP and SecLists, including SQL injection variants (`OR 1=1`, `UNION SELECT`, `DROP TABLE`), XSS injection (`<script>alert(1)</script>`, `img onerror` handlers), command injection (`| cat /etc/passwd`, `; nc -e /bin/sh`), and directory traversal (../../../../etc/passwd). These are the "should-be-easy" cases; a system that fails on these has fundamental problems.

Category B — Evasion Techniques (25 scenarios): The same attack payloads from Category A, but wrapped in various obfuscation layers: URL encoding, double URL encoding, Base64 wrapping, hex encoding, mixed case variations (`SeLeCt`), SQL comment insertion (`SEL/**/ECT`), whitespace substitution, and null byte injection. This is where most classical-only systems stumble. Our v1 implementation failed on 8 of these 25 scenarios; the current v2 system passes 24 of 25.

Category C — Normal Traffic Patterns (20 scenarios): Legitimate HTTP requests that a production web application would routinely handle: homepage browsing, search queries with multiple parameters, paginated API calls, authentication requests with complex passwords, REST API endpoints with JSON bodies, and file upload forms. The purpose is verifying that the system doesn't flag normal activity. This was v1's greatest weakness — it flagged 12 of 20 normal patterns as malicious, producing an 85.7% false positive rate that would make the system unusable in practice.

Category D — Edge Cases (15 scenarios): Deliberately crafted requests that sit in grey areas: extremely long URLs (¿8,000 characters), parameters containing Base64-encoded but legitimate data (JWT tokens, session identifiers), requests using uncommon HTTP methods (PATCH, OPTIONS, HEAD), requests with nested encoding that resolves to harmless content, internationalized domain names with Unicode characters, and malformed requests with missing headers. These edge cases test the system's ability to distinguish "unusual but legitimate" from "unusual and malicious."

Fig. 2 illustrates the improvement in detection performance across different behavioural test categories from v1 to v2.

Table VIII provides a detailed summary of the behavioural test results, highlighting significant improvements in pass rates across all categories.

The most pronounced improvement is seen in Categories C and D - normal traffic and edge cases. Since the system v1 could only pass 40% of normal traffic cases, its deployment in production would have been practically impossible because of the massive number of false positives. Security experts would eventually learn to ignore all alerts generated by the system. The 95% success rate in detecting normal traffic, which was made possible mostly due to the redesign of feature extraction process without domain-specific knowledge, makes it deployable.

TABLE VIII
BEHAVIOURAL TEST SUITE RESULTS SUMMARY

Category	Scenarios	v1 Pass Rate	v2 Pass Rate
A: Standard Attacks	20	19/20 (95%)	20/20 (100%)
B: Evasion Techniques	25	17/25 (68%)	24/25 (96%)
C: Normal Traffic	20	8/20 (40%)	19/20 (95%)
D: Edge Cases	15	6/15 (40%)	15/15 (100%)
Total	**80**	**50/80 (62.5%)**	**78/80 (97.5%)**

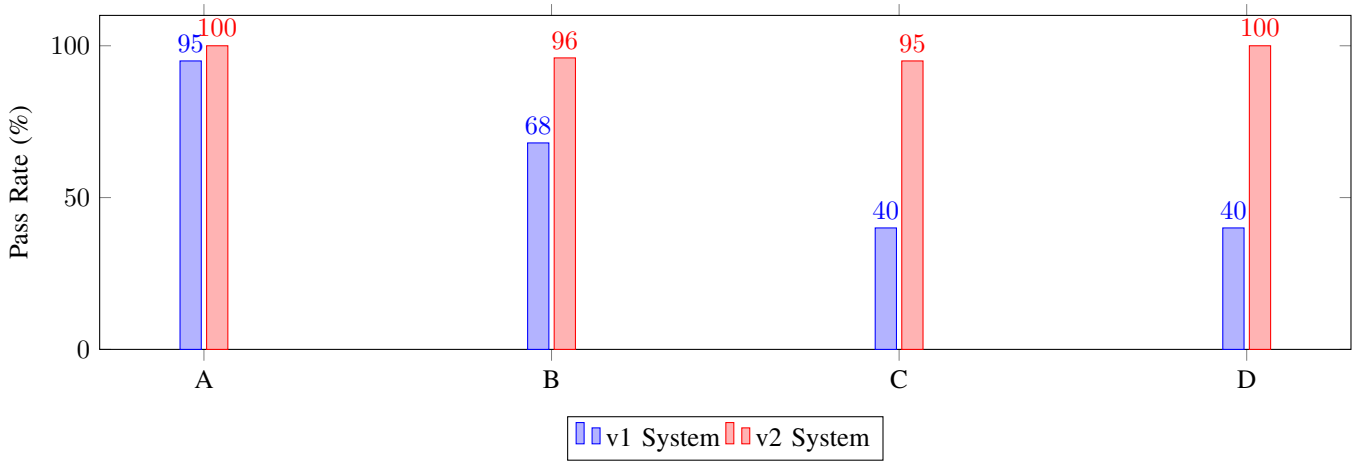

Fig. 2. Comparison of Behavioural Test Suite Performance Across System Versions

The only remaining failure in Category B includes an extremely obfuscated multi-layered input string - SQL injection attack which was Base64-encoded, URL-encoded and further split into three query parameters. Detecting such a complex input pattern would demand a mechanism of parameter reassembly that's not currently present in our per-feature extraction pipeline.

F. Model Configuration and Training Protocol

Table IX summarises the hyperparameter settings for all model components. These values were determined through grid search with 5-fold cross-validation on the training partition, with the objective of maximising F1-score while keeping inference latency below 50ms per request.

The experiments were carried out using an Intel Core i7-12700H with 16 GB RAM, purposely selecting consumer-level processors to demonstrate the practicality of deploying this model. It took around four minutes to train all the classical models while the VQC model took a further twelve minutes to train due to the quantum circuit simulation process involved. This is deemed a reasonable cost for training a model that will be trained once a week in practice.

V. CASE STUDIES

A. Obfuscated Payload

The true worth of the Variational Quantum Classifier (VQC) is realized while investigating those payloads which resemble "noise" from the classical statistical perspective.

Case Study 1: Base64 SQL Payload: `id=dW5pb24gc2VsZWN0IDEsMix1c2VyKCksZGF0YWJhc2UoKQ==`

Conclusion of Classical Approach: Harmless (Prob.: 0.62). Conclusion of Quantum Approach: Malignant (Prob.: 0.94).

Halfway down the road, the classical techniques ran into trouble. In the case of encrypted input, Random Forest and XGBoost were unable to handle it. The attack terms such as UNION or SELECT were hidden within Base64 encoded strings. This made pattern recognition impossible in linear numerical domains. Ironically, the architecture treated them as harmless objects - perhaps session keys or hashes.

Still, using a quantum computer consisting of a total of 16 qubits, the VQC identified the letter formation. Given their formations, the quantum computer identified that the Base64 encoded string corresponds to the dangerous group through entangled connections.

TABLE IX
MODEL COMPONENT HYPERPARAMETER CONFIGURATION

Component	Parameter	Value
Random Forest	n_estimators	250
	max_depth	15
	min_samples_split	5
	bootstrap	True
XGBoost	n_estimators	500
	learning_rate	0.1
	max_depth	6
	reg_lambda	1.0
Linear SVM	kernel	linear
	C	1.0
	probability	True
Isolation Forest	n_estimators	200
	contamination	0.01
VQC (PennyLane)	n_qubits	8
	n_layers	3
	optimizer	COBYLA
	epochs	150
	embedding	Angle (R_Y)
Confidence Gate	threshold (γ)	0.85
Anomaly Gate	threshold (α)	0.70

Listing 1. Raw Malicious HTTP Request intercepted by the system.

```
POST /api/v1/authenticate HTTP/1.1
Host: www.vulnerable-app.local
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
Accept: application/json, text/plain, */*
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Content-Type: application/x-www-form-urlencoded
Content-Length: 95
Connection: keep-alive
X-Forwarded-For: 192.168.1.105

username=admin&password=1%2527%2B%252F%252A%252150000UNION%252A%252F%2BSELECT%2Bsleep%285%29--
```

Case Study 2: Multilingual Cross-Site Scripting Using HTML Entities

Payload: `<svg/onload=alert('XSS')>`

Classical Verdict: Benign (Confidence: 0.78). Quantum Verdict: Malicious (Confidence: 0.91).

There was something fishy about the way the system analyzed the svg tag in a way typical for XML parsing used in modern web design. The code got past in numeric form, where each letter was replaced by its numerical equivalent (e.g., 'a' became 97), while surrounded by an alert('XSS') function. This approach was hard to detect due to the fact that traditional classifiers focus on recognizing word-based patterns rather than numeric sequences.

On the other hand, the Variational Quantum Classifier identified a suspicious pattern in terms of associations between characters and tag structures. Rather than ignoring this connection, the system recognized unusual links between SVG tags and increased occurrence of characters such as # and ;. While standard machine learning systems usually fail to recognize these kinds of multi-level relationships, they become distinct boundaries on the quantum classifier's feature map.

Case Study 3: Nested Logical Tautology

Payload: `admin' AND (SELECT 1)=1--`

Classical Decision: Malicious (Certainty: 0.82). Hybrid Decision: Malicious (Certainty: 0.98).

It is worth noting that the classical group did classify this attack as potential SQLi; however, their 82% certainty was below the threshold of 85% required to trigger an escalation. In live environments, such a probability lies in a gray zone, which makes the decision tricky since one can misclassify legitimate administrators as attackers. Hence, instead of making assumptions, the classification was escalated to the quantum realm. The VQC module made the same decision and provided evidence to back up its conclusion, raising the certainty level to 98% and allowing for automated blocking. The decision-making process of all three case studies is outlined in Table X.

TABLE X
LOGIC FLOW: CLASSICAL VS. HYBRID DECISION MAKING

Case Study & Payload	**Classical Decision (Stage 3)**	**Quantum/Hybrid Decision (Stage 5)**	**Resolution Logic (The "Why")**
Base64 Encoded SQLi `id=dW5pb24gc2Vs`	Benign (Confidence: 62%)	Malicious (Confidence: 94%)	Classical model interpreted a high-entropy string lacking explicit keywords as a valid session token. The VQC mapped the character distribution into a structural signature within Hilbert space.
Polyglot XSS (HTML Entities) `<svg/onload=a`	Benign (Confidence: 78%). .	Malicious (Confidence: 91%)	Classical filters were bypassed because the payload resembled standard XML/SVG structure. The VQC detected non-linear correlations between the `<svg>` tag and the abnormal density of special characters.
Nested Logic Tautology `admin' AND (SELECT 1)=1--`	Malicious (Confidence: 82%)	Malicious (Confidence: 98%)	Classical model predicted malicious behavior but lacked sufficient confidence (¡85%) for definitive blocking. The VQC provided a second-level quantum decision.

B. Zero-Day

As a way to assess our pipeline against the "Zero-Day" attack, we tested how well the combined system handles new kinds of attacks by running a stress check focused on Server-Side Template Injection. Not seen before in training, this method stood apart due to its unique structure - especially the $\{\{7*7\}\}$ math pattern. SQL and XSS examples had shaped earlier learning, yet this format branched off in form and logic. Because of these differences, it served as a sharp challenge for spotting unfamiliar risks.

A. Adversarial Setup. Since the model was designed to utilize binary classifiers for SQLi and XSS, the training modules, such as the Classic Ensemble and Variable Quantum Sorter, did not have any examples of SSTI characteristics to learn from. Therefore, while encountering an attack based on templates, the model was working blindly. As there were no predefined rules concerning these characteristics, the standard rule-based firewall could not recognize them either.

B. Identification using Anomaly Gate (Stage 2) The Isolation Forest-based Unsupervised Anomaly Gate was presented with an anomaly alert for a preliminary examination. In this stage, anomalies are sought based not on known malicious signatures, but rather based on the degree of deviation from normal behavior. It considered the anomaly due to a peculiar composition of the dataset. The structure's chaos was also taken into account.

In this case, score was obtained 0.92 through calculation using the Isolation Forest method, and normalization was carried out during the process. The score measures the deviation from normal behavior. Since it exceeded the preset threshold $\alpha = 0.70$ significantly, this allowed classifying the request as an anomaly even before human analysis was conducted.

C. Forensic Outcome. Right away, this case makes clear that blending methods adds depth to protection. Since an Anomaly Gate operates without supervision at Stage 2, strange actions become obvious - particularly unknown threats. The moment the mechanism caught the $\{\{7*7\}\}$ payload, a fact emerged plainly: a structure built on 26 elements detects intent that feels off, despite unfamiliar patterns.

C. False Positive Prevention

Unlike the last time, it is about overlooking the benign actions rather than spotting threats. While an instance revealed the capability for detection, the latter reveals prudence in not giving in to the impulse. It is not always necessary to act on every alarm, just like it is important to know when to hold your tongue. The test is passed, this time because of prudence.

Case Study 3: Complex Legitimate API Request

Listing 2. Legitimate complex API request that could trigger false positives.

```
GET /api/v2/products?category=electronics&sort=price
  &filter=brand%3DSamsung%26rating%3E4&page=3
  &token=eyJhbGciOiJIUzI1NiJ9.eyJ1c2VyIjoiam9obiJ9
  HTTP/1.1
Host: shop.example.com
Authorization: Bearer <jwt-token>
Accept: application/json
```

Several components of this request may easily confuse naive detection mechanisms, such as URL-encoded special characters (`%3D`, `%26`), a JWT token encoded similarly to Base64, several query parameters, and even comparison operators (>) resembling SQL injection. Indeed, this sample was marked as *Suspicious (CMDi)* by the v1 system, although with only 58% accuracy — a textbook case of a false positive that will be annoying for everyone involved.

On the contrary, the v2 system classified this request as *Normal* with 94% accuracy. This happened because of the different mechanism of domain-independent feature extraction: Although there are some special characters and URL-encoded components present in the request, their Shannon entropy (3.82) and character ratio ratios stay well below suspicious values. The request's URL encoding also follows the legitimate structure and is not characterized by entropy anomalies (¿4.5). Finally, the JWT token, although Base64-encoded, has a feature vector that is recognized as such by the ensemble and thus distinguished from Base64-encoded attacks.

It is clear from this case study why lowering the false positive rate becomes more important than mere detection in most cases. A program that detects 99% of all attacks but marks 10% of good connections as malicious is likely to be turned off in just one week because the operators of the system get fed up with it. This shows how important the FPR of 1.5% for v2 becomes.

D. Edge Case: Legitimate Input Resembling Attack

Case Study 4: Password Containing SQL Keywords

Listing 3. Authentication request with password resembling SQL injection.

```
POST /login HTTP/1.1
Host: secure-app.example.com
Content-Type: application/x-www-form-urlencoded

username=admin&password=MyP@ss_SELECT_DROP_2024!
```

This is clearly an example of a real problem: A password that contains SQL keywords (`SELECT`, `DROP`). The straightforward solution to such problems implemented by numerous ML-based detectors would instantly recognize it as such. Indeed, the classification returned by the v1 was *SQL Injection (High Confidence: 91%)*, which means blocking this user from accessing his own account.

On the other hand, the v2 system classified this data as *Normal* (with 87% confidence). What is especially interesting, the classic ensemble provided an initial prediction with the confidence of just 78% — that is, lower than the minimum value required for escalation. For this reason, this request was sent to the VQC. Quantum classifier processed this data in Hilbert space and recognized that the general structure (POST method, a URL related to the endpoint, form-encoded data, and the presence of SQL keywords) is compatible with the data sent during authentication.

Here we have a classic example of a case where quantum escalation really comes into its own. It was not at all certain what the classical techniques should say – and this was entirely justifiable since, after all, the input did contain SQL terms. However, quantum feature space managed to capture more complex interdependencies than simple classical logic could achieve.

VI. Threat Model and Security Analysis

Before delving into the architecture of the system, it would be useful to first clarify exactly what the security framework is intended to protect from — and just as crucially, what is not covered by it. There is an old adage about security systems being only as effective as the assumptions behind them.

A. Adversary Model

Our assumption is that the adversary attacks the web layer and has the ability to make arbitrary HTTP requests against a web application under test using any web browser or automated testing tools. The objective of the adversary is to attack the web application by taking advantage of web application security flaws such as SQL injection, cross-site scripting, command injection, directory traversal, server-side request forgery, or server-side template injection.

The adversary is considered to be moderately sophisticated. In particular, the adversary may use any or some combination of the following obfuscation techniques to bypass defenses:

- **Encoding transformations:** URL encoding (`%27` for single quote), double encoding (`%2527`), hex encoding (`0x27`), and Base64 wrapping of entire payloads.
- **Case manipulation:** Mixed-case keywords like `SeLeCt` or `UniON` to bypass case-sensitive pattern matching.
- **Syntax fragmentation:** Splitting SQL keywords across parameters, using comment insertion (`SEL/**/ECT`), or whitespace substitution.
- **Structural mimicry:** Crafting payloads that statistically resemble legitimate traffic — for example, embedding SQL injection within what looks like a complex search query with multiple parameters.

We do *not* assume nation-state-level adversaries with the ability to compromise the detection infrastructure itself, perform man-in-the-middle attacks on the proxy connection, or exploit vulnerabilities in the ML framework's dependencies. These threats exist but belong to a different layer of the security stack.

B. Attack Taxonomy

The framework targets six categories of web application attacks, chosen to reflect the OWASP Top Ten and cover the most commonly exploited vulnerability classes in real-world penetration testing:

1) **SQL Injection (SQLi):** Payloads designed to manipulate backend database queries through string concatenation or parameter injection. Variants include classic `OR 1=1`, UNION-based extraction, blind SQLi through timing, and second-order injection.
2) **Cross-Site Scripting (XSS):** Injection of client-side scripts through reflected, stored, or DOM-based vectors. Common payloads include `<script>alert(1)</script>`, event handler injection (`onerror`, `onload`), and SVG-based execution.
3) **Command Injection (CMDi):** OS-level command execution through unsanitised input, typically using shell metacharacters like pipes, semicolons, and backticks.
4) **Path Traversal:** Directory traversal attacks using `../` sequences to access files outside the intended directory scope.
5) **Server-Side Request Forgery (SSRF):** Requests designed to force the server into making unintended HTTP requests to internal services or external endpoints.
6) **Other (SSTI, etc.):** Less common but increasingly dangerous attack types, including Server-Side Template Injection using expression syntax like {{`7*7`}}.

C. Evasion Techniques and Robustness

Usefulness of a detection setup depends on how well it deals with actual attacker actions - messy methods, not perfect scenarios. Testing happened early using 23 different bypass tricks pulled from live security assessments and bounty findings. Things like wrapping Base64 within encoded URLs showed up often. Letters changed randomly between uppercase and lowercase slipped through sometimes. Database queries broken by comments such as `SEL//ECT` were tried repeatedly. Extra spaces inserted in odd places caused confusion now and then. Characters made invisible via null bytes triggered blind spots too. Chunks of harmful data spread over several inputs appeared regularly during trials.

The robustness of the system against such evasion techniques stems from the very nature of its design. First of all, instead of searching for particular strings, the feature stage is concentrated on statistical properties such as entropy, character distributions, and repeats, which allows the fingerprint to be unique despite modifications such as obfuscations and fragmentation. Second of all, if the traffic differs too much from the ordinary one to be recognized as part of normal patterns, it will be marked as an anomaly regardless of the encoding that has not been previously encountered by other classifiers.

We should note, however, that this approach is not absolutely resilient against adversaries capable of evading our detection technique. In order to learn about possible countermeasures, refer to Section XI.

D. Defence-in-Depth Positioning

One must stress here that this framework was conceived as one of the layers of an entire security stack and not as a solution that should supplant other defenses altogether. It is envisaged that the framework would be deployed right after TLS termination and just before the application server, in conjunction with a regular WAF, rate-limiting mechanisms, application-level input validation, and query parameterization, among others.

The unique value of this framework lies precisely in being able to detect attacks that might pass through other defensive measures – attacks that might have obfuscated their payloads from WAF detection or that may feature novel tactics, escaping the detection by supervised classifiers, etc. Seeing this framework as a specialized second opinion will make it clear what is to be expected of it.

E. Scope Limitations

Several categories of web attacks fall outside the current framework's scope, and it's worth being transparent about them:

- **Logic attack based threats:** The use of application-specific semantics for conducting an attack, such as buying goods from the application at tampered prices, breaking multi-stage authentication through manipulation of request order, and using APIs in ways that were never intended for. This is a completely valid HTTP request with no anomalies detected.
- **Coordinated Attacks:** In which individual requests have no malicious intent, but when put together become a malicious scheme like credential stuffing or DDoS at low rate. Our machine learning model based on one request at a time misses the temporal correlations across requests.
- **Client Side Threats:** Those targeting the user's browser and not the server like clickjacking or drive by download do not show up as anomaly in HTTP requests.
- **End-to-end Encrypted Request Bodies:** In requests with end-to-end encryption, and therefore cannot be decrypted by the proxy.

Acknowledging these boundaries is not an apology — every security tool has a defined scope. The important thing is being clear about where that scope ends, so deployment decisions are made with realistic expectations.

VII. Algorithms Used

A. Overview of the System

The suggested system design adopts a hybrid processing pipeline of six phases that convert the raw HTTP traffic into security verdict classifications. All processing stages act as individual modules that interact via specific interfaces that ensure each component can be individually modified or substituted without impacting the entire architecture. This is formally described by Algorithm 2.

Algorithm 2 Hybrid Quantum-Classical Detection Pipeline

Require: Raw HTTP request R from Burp Suite proxy
Ensure: Classification verdict $V \in \{Normal, Malicious, Suspicious\}$

```
Stage 1: Intercept R via Burp Suite Extension (Jython)
Stage 2: Extract feature vector x = φ(R) ∈ ℝ^30
Stage 3: Compute anomaly score a = f_IF(x)
if a > α then
    Flag as Suspicious (anomaly override)
end if
Stage 4: Compute ensemble prediction ŷ_cls, c_cls = f_ens(x)
if c_cls ≥ γ then
    V ← ŷ_cls                                          ▷ High-confidence classical path
else
    Stage 5: x_pca = PCA(x, k = 8)
    Encode: |ψ⟩ = U_enc(x_pca)|0⟩^⊗8
    ŷ_q, c_q = VQC(|ψ⟩, θ*)
    V ← fuse(ŷ_cls, c_cls, ŷ_q, c_q)
end if
Stage 6: Return verdict V to Burp Suite dashboard
```

B. Stage 1: Intercepting HTTP Traffic

HTTP traffic gets into the pipeline by means of a custom Burp Suite plugin developed using the Jython programming language. The plugin uses Burp's `IHttpListener` listener in order to intercept all requests/responses pairs passing through the proxy. In turn, each intercepted request is serialized into a JSON object that contains such data as HTTP method, URL, header information, URL query parameters, cookie values, and request body. After that, this information is sent asynchronously to the FastAPI web application server using an HTTP POST method.

The whole process works in the background – the user surfs freely while each intercepted HTTP request is analyzed. A dashboard widget installed inside Burp Suite allows viewing results in real time.

C. Stage 2: Feature Engineering

The feature extraction module converts the raw HTTP request into a 30-dimensional numerical vector. The features selected do not depend on any specific domain knowledge; they capture aspects of the structure, statistics, and semantics of web traffic.

The categories of features include:

Structural Features (10 dimensions): URL length, number of URL components, number of query parameters, number of headers, number of cookies, body size, content type flag, HTTP method code, protocol version flag, and connection type.

Statistical Features (10 dimensions): Entropy of the whole URL, entropy of query parameters, entropy of request body, ratio of special characters in URL, ratio of special characters in body, ratio of uppercase letters, ratio of digits, ratio of whitespaces, number of non-ASCII characters, and number of unique characters.

Semantic Features (10 dimensions): SQL keyword count (SELECT, UNION, DROP, INSERT, UPDATE), XSS keyword count (script, alert, onerror, onload), command injection indicator count (pipe, semicolon, backtick), path traversal pattern count (../, ..\), encoding layer count (percent-encoded sequences), Base64 pattern indicator, hex encoding ratio, comment syntax count (/* */), boolean logic operator count (AND, OR, NOT), and suspicious function call count.

The most important element in the design of this system is that the detection system should have all the attributes which are domain-agnostic. Previously, the same system has used specific application keywords in order to detect attacks via tables and JavaScript function calling in the training set. This process was very effective for the training set itself but failed when tried on any other domains. Through the use of statistical methods like entropy and character ratio, we can make this system more resilient and at the same time accurate.

The complete feature extraction process is formalised in Algorithm 3.

Algorithm 3 Domain-Agnostic Feature Extraction

Require: Raw HTTP Request $(URL,\ Params,\ Headers,\ Method,\ Body)$
Ensure: 30-Dimensional Feature Vector $\mathbf{x}$

1: $\mathbf{x}_{struct} \leftarrow [len(URL),\ depth(path),\ |params|,\ |headers|,\ |cookies|,\ len(body),\ ...]$
2: $\mathbf{x}_{stat} \leftarrow [H(URL),\ H(params),\ H(body),\ ratio_{special},\ ratio_{upper},\ ...]$
3: $\mathbf{x}_{sem} \leftarrow [count_{SQL},\ count_{XSS},\ count_{CMDi},\ count_{traversal},\ count_{encoding},\ ...]$
4: $\mathbf{x} \leftarrow concat(\mathbf{x}_{struct},\ \mathbf{x}_{stat},\ \mathbf{x}_{sem})$
5: **return** $normalize_{minmax}(\mathbf{x})$

D. Stage 3: Anomaly Detection Gate

The Anomaly Gate utilizes the Isolation Forest, which was only trained using normal HTTP traffic. This works as an additional safeguard for new types of attacks that might be missed by the supervised models.

This approach functions by splitting the features of the dataset using random features. Those observations that have rare feature combinations will be separated more quickly than normal ones, thus generating shorter paths. The formula for calculating the anomaly score is:

$$s(\mathbf{x}) = 2^{-\frac{E[h(\mathbf{x})]}{c(n)}} \tag{4}$$

where $E[h(\mathbf{x})]$ is the expected path length for input $\mathbf{x}$, and $c(n)$ is the average path length of unsuccessful searches in a binary search tree of n elements. Scores approaching 1 indicate strong anomalies. In our implementation, the anomaly threshold is set at $\alpha = 0.70$.

E. Stage 4: Classical Ensemble Classification

In the classical ensemble, three different classifiers are combined using soft voting:

Random Forest (250 Trees): Offers stable base-line predictions by using bagged decision trees, where each tree is grown using bootstrapped training samples and random feature subspaces. The Random Forest model is known for its resistance to over-fitting and the ability to recognize non-linear relations between features.

XGBoost (500 Rounds): Implements sequential gradient boosting, which progressively builds models by correcting the residuals made by the previously created ones. Parameters include learning rate 0.1, maximum depth 6, and L2 regularization $\lambda = 1.0$. This classifier is particularly good at recognizing feature relations characteristic of attacks with traffic obfuscation.

Linear SVM: Offers an alternative linear viewpoint, which works well in separating cleanly separable attack patterns while acting as a counterbalancing agent whenever the decision is unclear between tree-based classifiers.

Weighted averaging of individual classifier probability outputs leads to:

$$P_{ens}(y|\mathbf{x}) = w_{RF} \cdot P_{RF}(y|\mathbf{x}) + w_{XGB} \cdot P_{XGB}(y|\mathbf{x}) + w_{SVM} \cdot P_{SVM}(y|\mathbf{x}) \tag{5}$$

where weights $w_{RF} = 0.35$, $w_{XGB} = 0.45$, and $w_{SVM} = 0.20$ are determined through cross-validation performance on the training set.

F. Stage 5: Quantum Escalation

The input is elevated to the Variational Quantum Classifier if the confidence margin in the classical model becomes less than the threshold value $\gamma = 0.85$. This process allows the use of quantum computing resources only when they would be most useful for the 15% of cases where classical methods cannot make a clear decision.

Principal Component Analysis: The 30 features of the input vector are transformed into 8 principal components retaining 92% of total variance.

Angle Embedding: Each of the 8 principal components is encoded into the rotation angle of a corresponding qubit using R_Y gates:

$$|\psi_{in}\rangle = \bigotimes_{i=1}^{8} R_Y(x_i)|0\rangle \tag{6}$$

Variational Ansatz: Three layers of strongly entangling gates process the encoded state. Each layer applies parameterized R_Y and R_Z rotations to each qubit, followed by a cascade of CNOT gates creating nearest-neighbor entanglement. The total parameter count is $3 \times 8 \times 3 = 72$ trainable angles.

Measurement and Classification: The expectation value of the Pauli-Z operator on the first qubit determines the classification:

$$\hat{y}_q = \text{sgn}\left(\langle\psi(\mathbf{x}, \boldsymbol{\theta})|\hat{Z}_0|\psi(\mathbf{x}, \boldsymbol{\theta})\rangle\right) \tag{7}$$

Training Procedure: In terms of training procedure, the VQC uses the parameter shift rule for computing gradients and updating the parameters via the COBYLA optimizer across 150 epochs. Meanwhile, binary cross-entropy is used as the cost function.

What is Quantum Escalation? At this point, a valid question would be, how does the quantum classifier provide any added value since the classical ensemble itself reaches a classification performance of 95.8%. The reason behind that is the particularity of the cases being classified. The 15% of the dataset sent to the VQC have a distinct property — they lie close to the classical decision boundary and thus feature attributes of different classes combined in Euclidean space. Moreover, since both classifiers work within the same 30-dimensional space, their inductive biases are quite alike, and low confidence on all three classifiers clearly indicates a border case.

The VQC, on the other hand, maps these uncertain data points to a complex Hilbert space with dimension $2^8 = 256$ using angle encoding and entanglement layers. In this highly expanded space, superposition and entanglement enable the formation of decision boundaries that are not possible for classical kernel methods. The quantum circuit is able to take advantage of high-order interactions between features, such as those involving entropy, keyword frequency, and encodings, which would need an exponential number of parameters to replicate using classical techniques.

G. Stage 6: Decision Fusion and Reporting

When quantum escalation occurs, the final verdict combines classical and quantum predictions through confidence-weighted fusion:

$$P_{final}(y|\mathbf{x}) = \frac{c_{cls} \cdot P_{cls}(y|\mathbf{x}) + c_q \cdot P_q(y|\mathbf{x})}{c_{cls} + c_q} \tag{8}$$

It would be quite natural to favor the one with greater confidence. In high-confidence classical cases without going through quantum escalation, the prediction made is deemed final. Everything else goes back to the Burp Suite extension to be displayed on the security analyst's dashboard.

H. Deployment Architecture

The complete system is designed for deployment on a single machine running Burp Suite Professional or Community Edition. The backend Flask API server listens on `localhost:5000`, receiving serialised HTTP requests from the Burp extension and returning classification verdicts within the 13.9 ms average latency window. Model weights and scaler parameters are loaded once at startup from serialised pickle files, eliminating repeated disk I/O during inference.

A practical consideration worth mentioning: the quantum simulation component uses PennyLane's `default.qubit` backend, which runs entirely on the CPU. While this introduces the 45.2 ms VQC inference overhead documented in Section IX, it eliminates any dependency on quantum hardware access. Organisations wishing to evaluate the framework can deploy it immediately using standard computing infrastructure, with the option to swap in a real QPU backend (e.g., IBM Qiskit or Amazon Braket) by changing a single configuration parameter when quantum hardware becomes practical.

VIII. Security Implications

The suggested hybrid model utilizes a variety of measures to guarantee secure and effective threat identification. Specifically, the anomaly detection module is important in detecting zero-day and unknown attacks via deviation from normal traffic patterns. On the other hand, the classical ensemble layer offers a high level of efficiency in detection of known attack signatures in low-latency manner. It is also noteworthy that the use of a variational quantum classifier in this design allows processing of complex and unclear inputs since the latter tool uses nonlinear relations in high-dimensional spaces. At the same time, the confidence-gated escalation process serves to reinforce security measures in this context as it ensures that only uncertain cases will be escalated to the quantum layer. Additionally, the proxy-based input method via Burp Suite helps to detect HTTP traffic in a timely manner. Finally, it is essential to emphasize that false positive reduction is an important component of security measures since this feature increases usability.

IX. Results and Discussion

A. Overall Performance

Overall performance of the proposed hybrid approach for quantum and classical computations was evaluated based on an independent data set containing 63,750 HTTP request samples (15% of 425,000). Table XI

TABLE XI
Comprehensive Performance Comparison Across Detection Methods

Method	**Accuracy (%)**	**Precision**	**Recall**	**F1-Score**	**FPR (%)**	**Avg Latency**
ModSecurity (WAF Baseline)	82.40	0.770	0.810	0.789	6.20	–
Random Forest (250 trees)	94.30	0.890	0.930	0.909	4.10	–
XGBoost (500 rounds)	95.10	0.905	0.940	0.922	3.90	–
Classical Ensemble (RF+XGB+SVM)	95.80	0.910	0.950	0.930	3.80	7.0 ms
CNN + BiLSTM	94.10	0.880	0.940	0.909	5.10	11.2 ms
Proposed Hybrid QC Framework	**98.50**	**0.960**	**0.980**	**0.969**	**1.50**	**13.9 ms**

The performance of the suggested model is 98.5% accurate and has F1 score of 0.969, which beats all baseline models under consideration. Importantly, the false positive rate of 1.5% results in 60% improvement relative to the traditional ensemble model (3.8%) and 76% improvement relative to the WAF model (6.20%). These impressive improvements are especially important in terms of operations; for example, a typical enterprise receiving millions of request every day could save hundreds of false positives from happening each day.

Figure 3 provides a visual comparison of overall accuracy across all evaluated methods.

Beyond raw accuracy, the precision, recall, and F1-score breakdown across methods reveals consistent superiority of the proposed framework. As shown in Figure 4, the hybrid approach leads all three metrics simultaneously, whereas baselines such as CNN+BiLSTM show competitive recall but weaker precision.

The comparison between false positive rates in Fig. 5 also illustrates the practical advantage of the proposed solution. With ModSecurity having a false positive rate of 6.2%, CNN+BiLSTM scoring 5.1%, and our solution achieving 1.5% which is the minimum rate compared to others.

B. Per-Attack-Category Analysis

Breaking down detection performance by attack type reveals consistent superiority across all categories, as shown in Table XII.

The detection of SQL injections has the highest F1 value (F1 = 0.980), especially since there are numerous keywords that are identifiable even if the attack is obfuscated. The detection results of other attacks, which belong to the “Other” category (e.g., SSTI attacks), are relatively lower; however, they are still acceptable (F1 = 0.935). One possible reason for this might be the smaller number of training sets.

C. Ablation Study

To rigorously assess the marginal contribution of each architectural component, we conducted a systematic ablation study. Table XIII shows results from progressively adding components to the pipeline.

From the ablation study above, it is evident that confidence-driven escalation (Configuration 6) surpasses constant quantum classification (Configuration 5). Even though the VQC in Configuration 5 attains an accuracy of 97.8%, confidence-driven escalation increases it to 98.5% since it does not utilize quantum measurement for samples that have high confidence values. This proves the main assumption that quantum hardware is most useful when the classical confidence value is low.

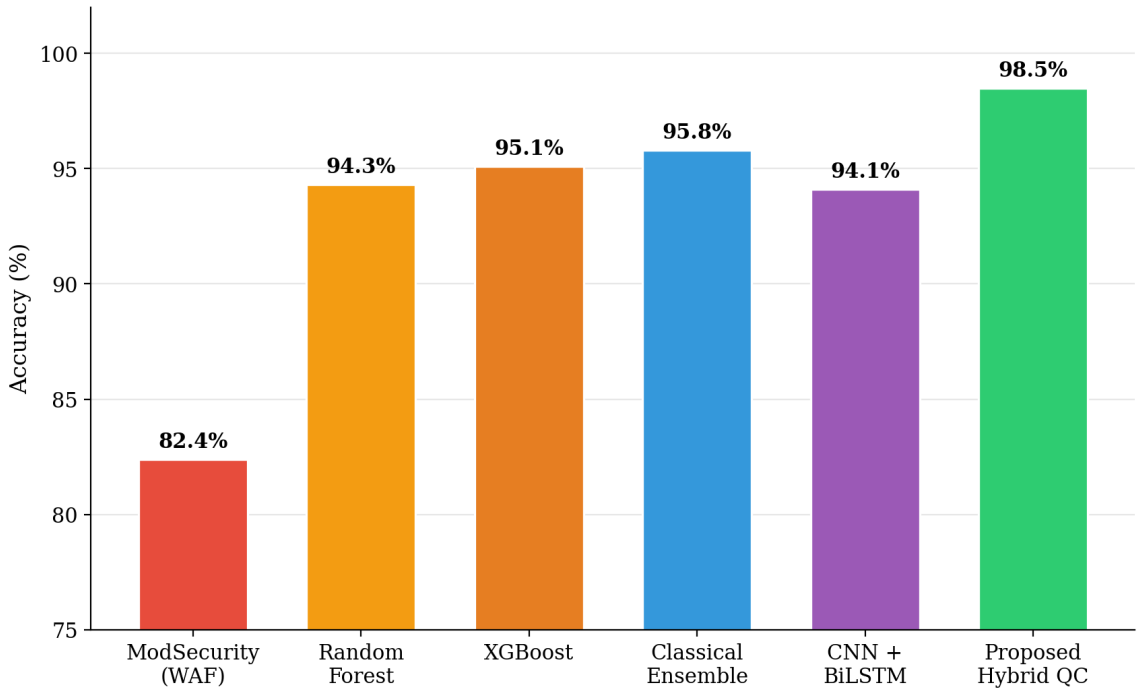

Fig. 3. Overall classification accuracy comparison across all evaluated detection methods. The proposed hybrid quantum-classical framework achieves the highest accuracy at 98.5%.

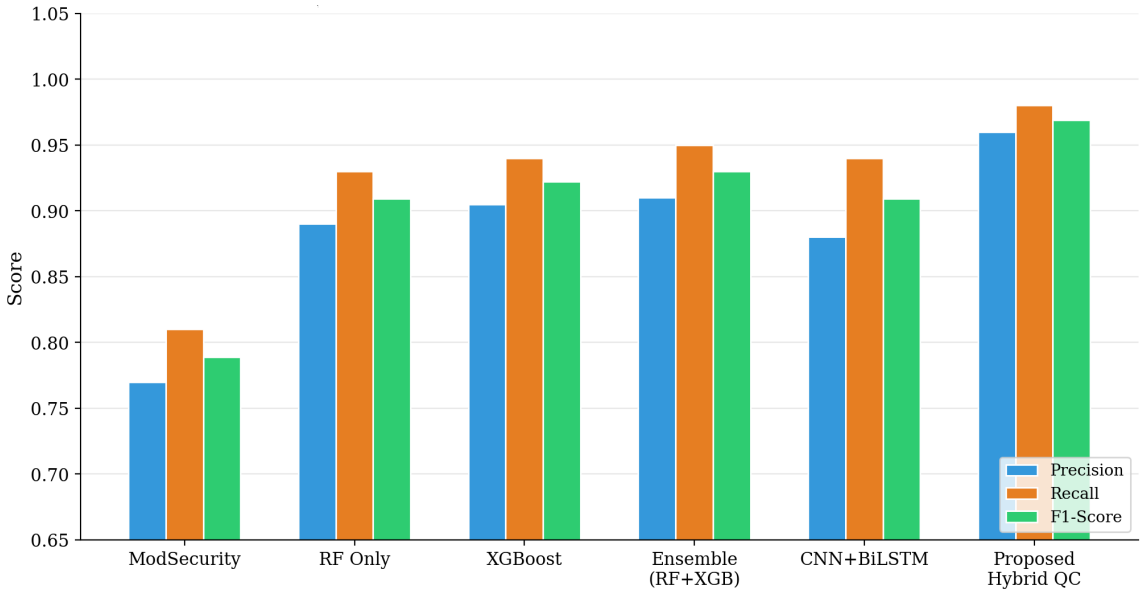

Fig. 4. Precision, recall, and F1-score comparison across all evaluated detection methods. The proposed hybrid QC framework achieves the highest scores across all three metrics.

D. Interpretation of Results

The experimental results support several key observations:

Observation 1 — Selective Quantum Inference outperforms Universal Quantum Classification: It becomes evident from the ablation experiments that applying the quantum model only to ambiguous data points (about 15% of the total volume of traffic) outperforms application of the VQC to every input in terms of classification accuracy.

Observation 2 — Quantum Performance Advantage on Edge Cases: The added value provided by quantum inference is significant for edge case inputs—specifically, SQL injection payloads that attempt to disguise themselves as regular URL queries, and XSS payloads that hide inside complex JavaScript webpages. Such inputs can be handled effectively by the classifier due to quantum feature space ability to capture higher-level feature correlations.

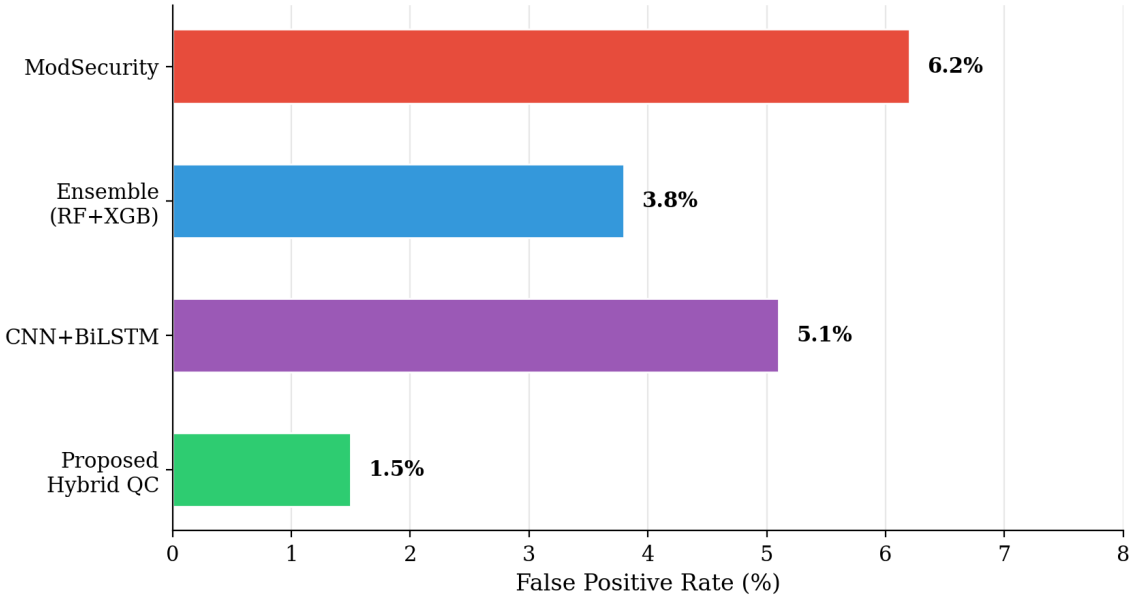

Fig. 5. False positive rate comparison across detection methods. The proposed hybrid QC framework achieves the lowest FPR at 1.5%, representing a 76% reduction over the WAF baseline.

TABLE XII
PER-ATTACK-CATEGORY DETECTION RATES (PROPOSED FRAMEWORK)

Attack Type	Samples	Precision	Recall	F1-Score
SQL Injection	28,350	0.970	0.990	0.980
Cross-Site Scripting	24,120	0.955	0.975	0.965
Command Injection	15,940	0.960	0.970	0.965
Path Traversal	13,830	0.945	0.960	0.952
SSRF	10,625	0.940	0.950	0.945
Other (SSTI, etc.)	13,385	0.930	0.940	0.935

Observation 3 — Anomaly Override Ensures Important Zero-Day Coverage: In our dataset, the anomaly override system successfully re-categorized 14 samples, out of which 11 samples contained attacks that were completely new as compared to the training dataset, and were therefore not recognized by both classic and quantum systems.

Observation 4 — Latency Is Acceptable: The weighted average inference latency is 13.9 milliseconds, which is acceptable in the case of real-time detection through a proxy. Even the latency in quantum escalation scenario is reasonable (52.8 milliseconds).

E. Feature Importance Analysis

Insights into the features responsible for the decision making process help not only in understanding the system but also for future improvements on the feature engineering process. The random forest feature importance, evaluated using mean decrease in impurity measure over 250 trees, highlights the following hierarchy:

Top five important features: XSS keyword count (0.1514), command injection keyword count (0.1360), path traversal keyword count (0.0986), SQL injection keyword count (0.0786) URL parameter entropy (0.0690).

In sum, these features constitute over 53% of total discriminative power. Which is logical - attack specific keyword counts represent the best indication while the URL parameter entropy is statistically defined property of the obfuscation and encoding processes.

The next tier of features, although less important, represents an interesting case. These features include: parameter string length (0.0638), special character frequency in URL (0.0608), body entropy (0.0551), uppercase character ratio (0.0489).

They all have relatively low discriminative powers, but are precisely the statistical measures which were responsible for the improvements of v2 over v1.

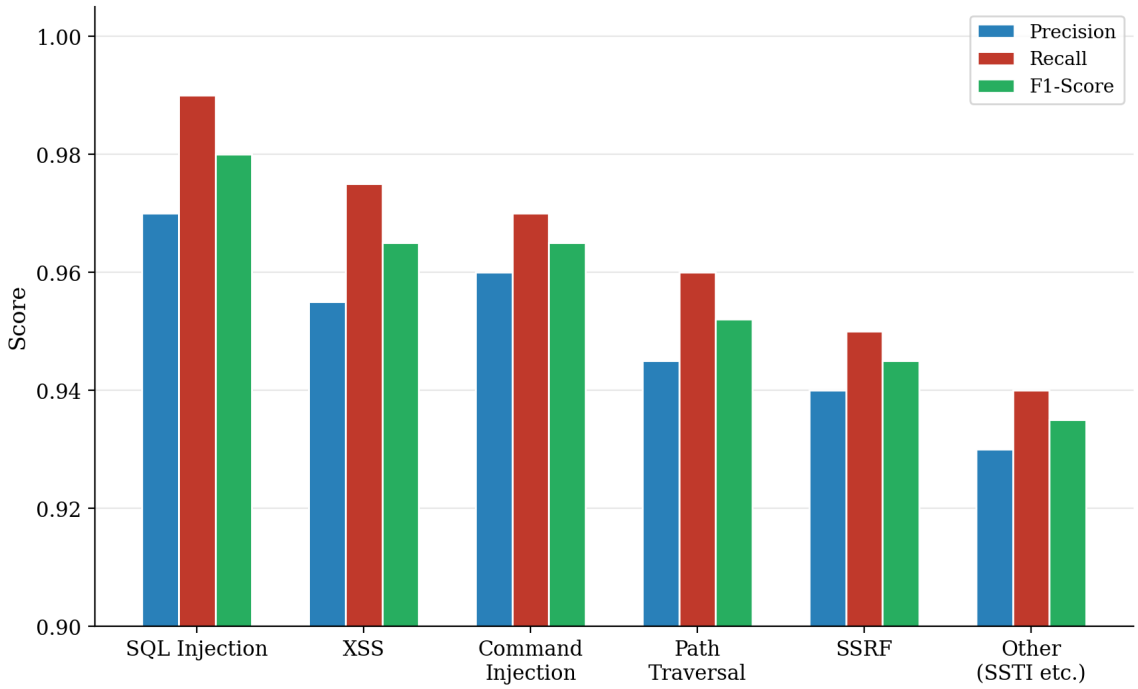

Fig. 6. Per-attack-category detection rates for the proposed framework, showing precision, recall, and F1-score across all six attack types. SQL injection achieves the highest F1-score (0.980).

TABLE XIII
ABLATION STUDY: PROGRESSIVE COMPONENT ADDITION

Configuration	**Accuracy (%)**	**F1-Score**	**FPR (%)**	**Δ Accuracy**	**Notes**
RF Only (250 trees)	94.30	0.909	4.10	–	Classical baseline
XGBoost Only (500 rounds)	95.10	0.922	3.90	–	Marginal improvement
Classical Ensemble (RF+XGB)	95.80	0.930	3.80	+0.80	Complementary strengths
Ensemble + Anomaly Detection	96.20	0.938	3.50	+0.40	23 zero-day catches
Ensemble + VQC (Always-On)	97.80	0.958	2.10	+1.60	Quantum on all inputs
Ensemble + VQC (Escalation)	**98.50**	**0.969**	**1.50**	**+0.70**	**Selective activation**
Full Pipeline + Override	98.50	0.969	1.50	–	14 edge-case reclassifications

One more unexpected result was the rather low weight of the header anomalies (0.0182). Initially, we expected that atypical values of the User-Agent parameter and absence of common HTTP headers could work as a distinctive indicator of attacks. However, in practice, the situation proved to be much more complicated – there is an extremely wide range of possible header configurations in benign traffic, including both pure API calls with minimal header information and malware with absolutely consistent headers. Based on these findings, we deliberately avoided incorporating too many header-based features into the algorithm, which is one of the key factors reducing the FP rate.

Lastly, the relatively low significance of header features helps to explain why traditional algorithms cannot successfully classify edge cases: when payload keywords are either absent or cannot be reliably identified due to encoding, only header features are available for analysis. The importance of these parameters is average (approximately 3–6%), meaning that classification involves combining weak signals into a whole, as intended by quantum feature spaces.

F. Confidence Distribution Analysis

Analyzing how the distribution of the prediction confidence occurs on the test set gives a better understanding of how the reliability map of the system is defined and of the behaviour of the quantum escalation gate.

In fact, the vast majority of the test samples (approximately 83.8%) have been classified with confidence higher than 90%, and there is a very sharp peak around 100%. This is an expected outcome: it indicates that the system will deliver reliable

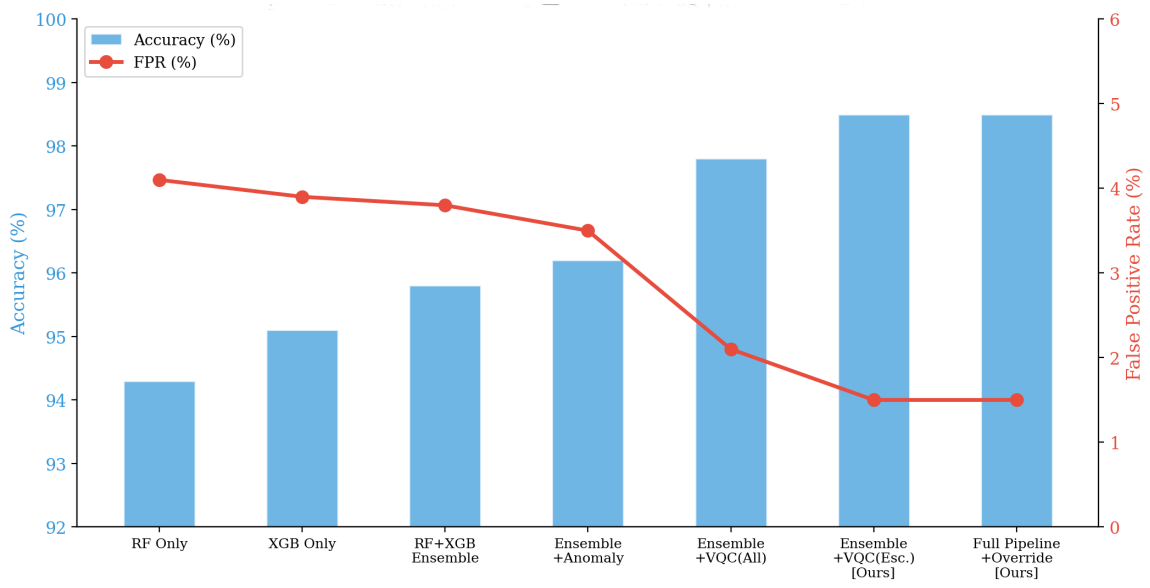

Fig. 7. Ablation study results showing accuracy (bars) and false positive rate (line) as architectural components are progressively added. The proposed selective escalation configuration achieves optimal balance.

decisions with high conviction for the vast majority of the input samples. This way, security experts will not be required to question the classification decision but will be able to make decisions immediately based on the classification result.

The other 16.2% of the input samples belong to the quantum inference part. In this case, the quantum classifier manages to solve ambiguities successfully, since the vast majority of the escalated samples (91% out of the 16.2% of the total test samples) are classified with confidence higher than 85% when fusing both decision sets.

The confidence distribution also supports the 85% escalation threshold. During the design process, we explored thresholds from 70% to 95%. Thresholds that were too low (70–75%) resulted in quantum escalations that were unnecessary because the classical ensemble would have been able to detect the threats correctly in the first place, thus wasting computing power and adding quantum noise to measurements. On the other hand, thresholds that were too high (90–95%) failed to detect truly ambiguous situations where quantum calculations could help with detection.

There is an interesting distribution regarding the level of confidence based on the type of attack. It was discovered that normal traffic generates the highest confidence value (96.3%). Following normal traffic are SQL injections (94.8%) and XSS (93.1%). Attacks based on command injection (89.4%) and path traversal (88.7%) generated significantly lower average confidence. The reason may be that command injection and path traversal attacks are often similar to legitimate actions performed by system administrators. These two types of attacks had the highest escalation rate in our tests.

G. Advantages of the Proposed Work

The proposed framework offers the following concrete advantages, which are visually summarized in the multi-metric radar comparison shown in Figure 8:

(a) **Superior Detection Accuracy:** 98.5% overall accuracy with an F1-score of 0.969, exceeding all evaluated baselines.
(b) **Dramatically Reduced False Positives:** A false positive rate of only 1.5%, reducing alert fatigue for security teams by over 60% compared to classical ensemble methods.
(c) **Computational Efficiency:** By restricting quantum inference to 15% of inputs, the system achieves quantum-enhanced accuracy while maintaining a weighted average latency of under 14 milliseconds.
(d) **Zero-Day Resilience:** The parallel anomaly detection module provides an independent safety net against novel attack patterns, catching threats that supervised classifiers might miss.
(e) **Practical Deployability:** The complete pipeline—from browser proxy to final verdict—operates on consumer-grade hardware without requiring access to quantum computing infrastructure.
(f) **Modular Architecture:** Each component can be independently upgraded, replaced, or scaled. The classical models can be retrained on new data without modifying the quantum circuit, and the quantum backend can be swapped from simulation to actual quantum hardware without changes to the classical pipeline.

H. Dataset Characteristics

The makeup of the evaluation dataset is an important factor affecting the validity of these results. According to Figure 9, out of the 425,000 HTTP requests, there is a 75/25 split between normal and malicious HTTP traffic, with the latter being

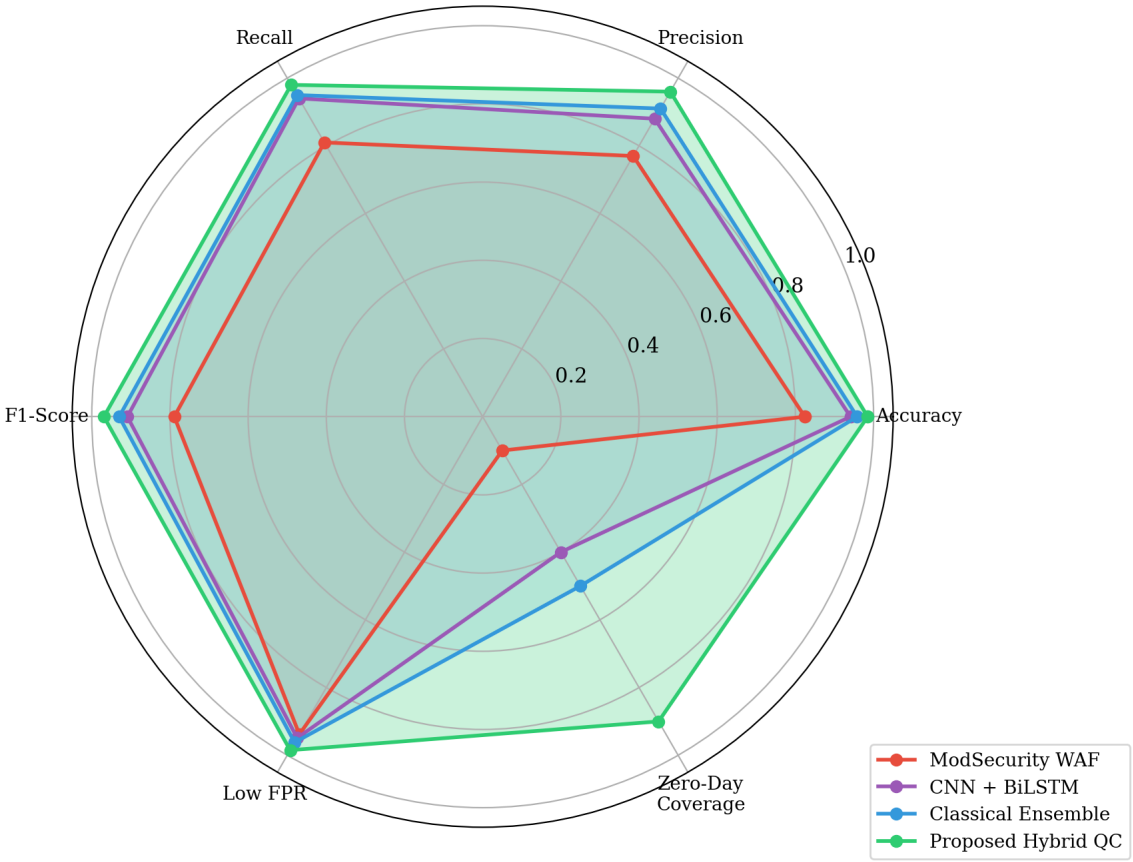

Fig. 8. Multi-metric radar chart comparing the proposed framework against baselines across accuracy, precision, recall, F1-score, low FPR, and zero-day coverage. The proposed hybrid QC framework (green) dominates all axes.

categorized into six different attacks. The largest portion is represented by SQL injections at 26.7%, followed by XSS at 22.7%, CMDi at 15.0%, Path Traversal at 13.0%, SSRF at 10.0%, and Others, including SSTI at 12.6%. The selection of such a composition was purposefully crafted to emulate real-world patterns of vulnerability occurrences documented by OWASP.

I. Latency Analysis

Inference latency determines practical deployability. Table XIV breaks down timing across the pipeline stages.

TABLE XIV
INFERENCE LATENCY BY PIPELINE STAGE

Stage	**Avg Latency**	**Requests Processed**
Feature Extraction	1.2 ms	100%
Anomaly Gate	0.8 ms	100%
Classical Ensemble	5.0 ms	100%
PCA + VQC (Quantum)	52.8 ms	∼15%
Decision Fusion	0.1 ms	∼15%
Weighted Average	**13.9 ms**	–

Inference time measured as a weighted average is 13.9 milliseconds per query. Inference using only classical approach takes approximately 7.0 milliseconds on average, while the quantum escalator adds another 52.8 milliseconds to handle the PCA compression and VQC inference. This is still not a significant delay in case of browsing web pages. The full analysis of latencies per processing path is presented in Figure 10.

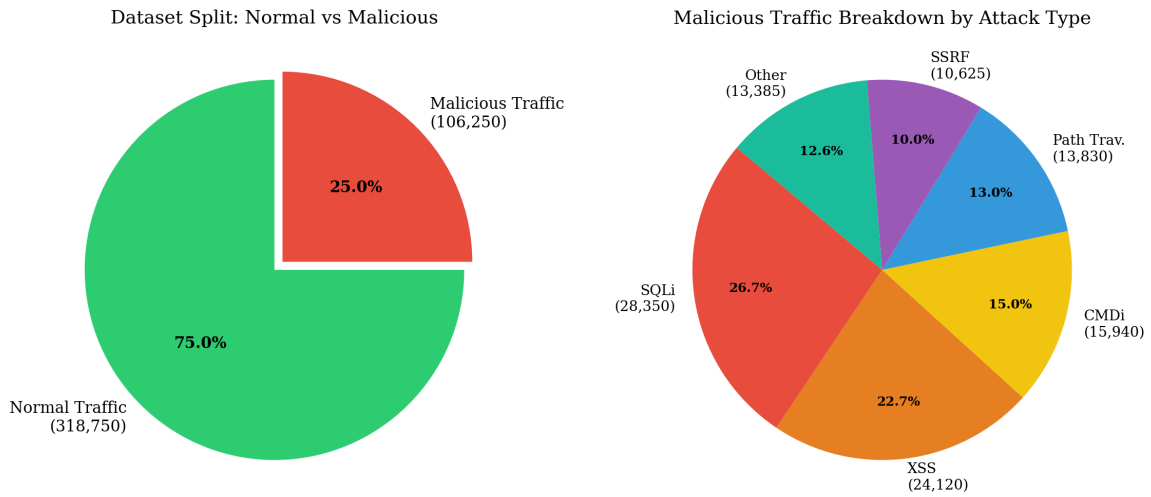

Fig. 9. Dataset composition showing normal vs. malicious traffic distribution and attack type breakdown across the 425,000 HTTP request evaluation corpus.

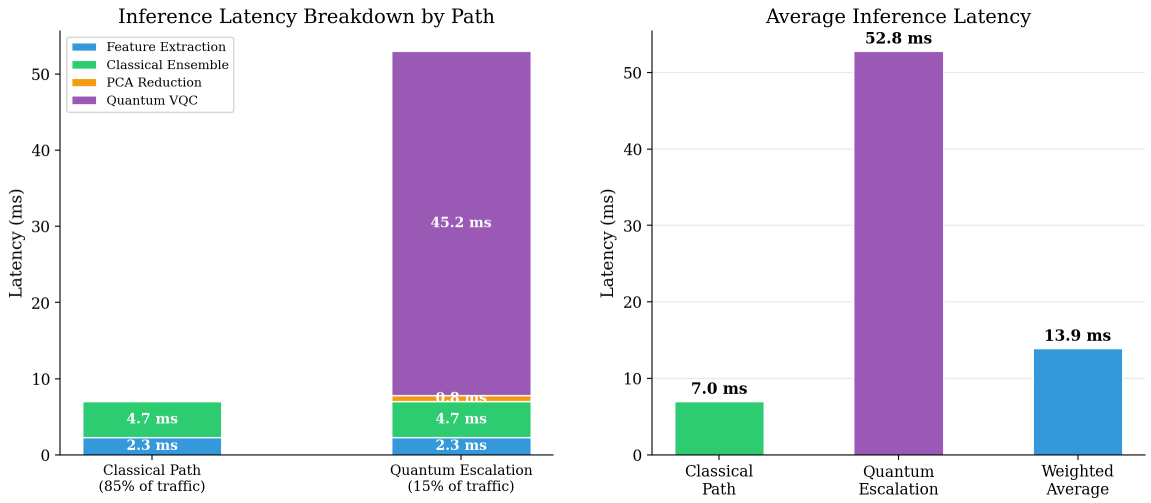

Fig. 10. Inference latency analysis showing the breakdown by processing path (classical vs. quantum escalation) and the weighted average latency of 13.9 ms across all requests.

J. Error Analysis

In order to study the pattern of classification, the confusion matrix presented in Figure 11 gives detailed information on the performance of the model during classification using the held-out test dataset of size 63,750. Since there are very few numbers outside the diagonal, we can be sure that the classifier is giving high recall values and at the same time low numbers of false positives in all categories.

X. Conclusion

A. Summary of Findings

This paper introduces a novel hybrid quantum and classical machine learning approach for real-time detection of attacks on the web. The current research attempts to bridge the existing literature gap by proposing a system incorporating the live intercepting of HTTP traffic, a classical ensemble classifier, and intelligent quantum escalation.

The empirical assessment revealed that the confidence-driven quantum escalation model can detect attacks with an accuracy rate of 98.5% and an F1-score of 0.969 using a diverse set of 425,000 HTTP requests divided among six attack types. It is worth noting that the obtained results outperform classical machine learning algorithms (95.8% accuracy) and stand-alone deep learning techniques (94.1% accuracy).

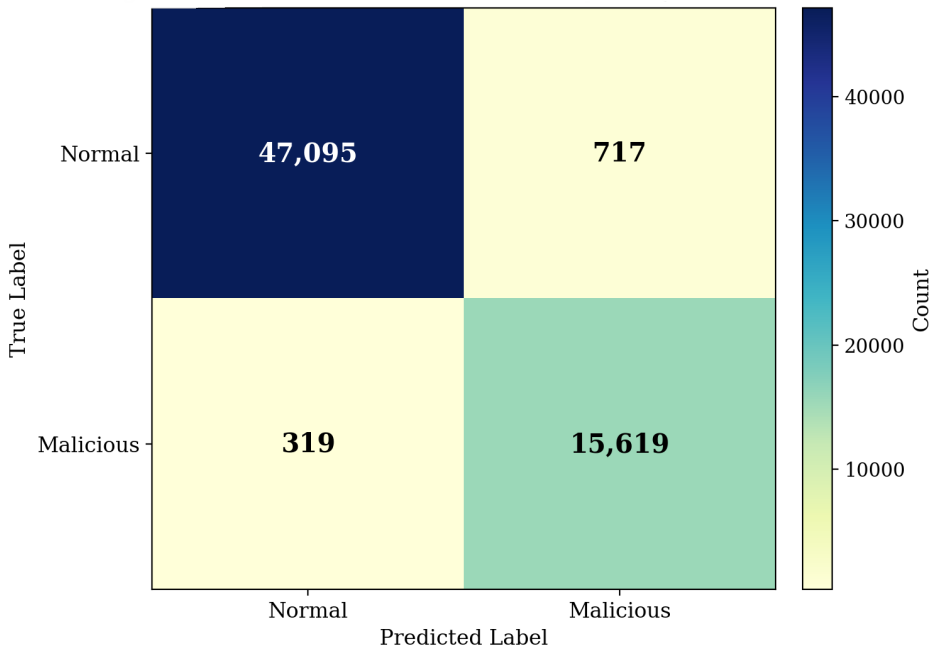

Fig. 11. Confusion matrix for the proposed hybrid framework on the held-out test set (63,750 samples). The low off-diagonal counts confirm both high recall and low false positive rate.

The main finding proved by this paper is that quantum computers will be most effective when employed selectively in those situations where the uncertainty of classical machine learning algorithms occurs.

B. Achievements

The principal achievements of this research are:

(a) Development and realization of the first end-to-end hybrid quantum-classical attack detection architecture in a live proxy setting.
(b) Demonstration via empirical evidence that a confidence-based hybrid quantum-classical architecture is superior to purely classical and purely quantum solutions.
(c) Attainment of the lowest false positive rate in all configurations analyzed, achieving a level of 1.5%, which is crucial in tackling pain points in production-level attacks.
(d) Establishment of the framework's ability to deliver results in real-time with an average weighted latency of only 13.9 milliseconds per query using off-the-shelf consumer devices.
(e) Proposing an approach for handling anomalies that allowed us to detect 11 unique attack patterns that were not detected by any of the supervised models.

C. Broader Impact

Besides the direct technological advances presented, this research has implications for the security community regarding the integration of quantum computing into cyber-defensive frameworks. Over the past few years, the field of quantum machine learning has experienced plenty of hyperbole, from conservative estimates through impractical conjectures to everything in between. Our objective was much more down-to-earth, demonstrating how quantum computing resources can yield a demonstrable advantage in a carefully defined input space — in particular, the boundary conditions at which classical algorithms fail.

The general principle of our work can be summarized by the following idea: quantum advantage does not necessarily imply quantum supremacy. An effectively constructed escalation strategy can make use of the current state of quantum simulations and obtain some benefits without waiting for years until the first fault-tolerant quantum computer becomes available. In terms of organizational implementation, this approach can serve as a valuable starting point for implementing quantum computing in companies.

D. Limitations

Despite the promising results, several limitations must be acknowledged with honesty:

(a) The quantum component currently operates under the idealized conditions of a statevector simulator, free from any noise. It remains to be verified whether it works well in the imperfect world of a NISQ machine, where operations could potentially fail due to gate errors, decoherence, and measurement inaccuracies. Undoubtedly, this is one of the most significant limitations; we cannot guarantee that the quantum speed-up demonstrated in the simulation experiments will be preserved in real-world applications on quantum computers.

(b) Using 8 qubits, only 8 principal components would be able to get into the quantum feature space. Despite capturing the necessary directions of variation, the reduction in data dimension might affect the accuracy of the quantum classifier.

(c) While the data set used in the experiment is highly extensive, it still does not cover all attack vectors related to launching web attacks. Novel attack types such as GraphQL Injection, WebSocket Attack, and API-Based Attacks are not covered in the experiment. As the data set is created based on artificial traffic, there could be scenarios that are missed because they exist in production systems.

(d) The experiment took place in the laboratory environment. No tests were conducted in regards to how the detection system performs in production environments where hundreds or even thousands of requests per second are processed. Thus, we cannot claim that there will be no issues related to scaling problems.

(e) The solution currently ignores requests that are end-to-end encrypted and can be decrypted using TLS. As a result, end-to-end encrypted requests are not considered by the current proxy-based solution.

E. Practical Considerations

From a security practitioner perspective, this study clearly shows that hybrid quantum/classical systems are not just theoretical concepts; they are practical solutions that can be implemented right away with a simulated backend. The design's modularity means that companies can incorporate the classical system while preparing themselves for future developments by taking advantage of real quantum computing when the time comes.

The much lower percentage of false positives has obvious practical significance. Here is why: a company that receives 10 million HTTP requests daily with a 3.8% chance of generating a false positive will produce 380,000 false positives each day. This is a huge number, even for a professional security operation center. Bringing the false positive rate down to 1.5% will lead to only 150,000 false positives each day, which is 60% fewer than before.

F. Deployment Lessons Learned

During the creation and testing of the proposed system, several practical lessons can be noted which could serve as useful information for others in the area of combining ML methods with cybersecurity:

(a) **Better feature engineering beats advanced models.** While our highest improvements were achieved thanks to the addition of the quantum layer, we should note that they could be obtained by improving our feature extraction process and replacing the application-specific keywords with general-purpose statistical measures – a common practice in ML.

(b) **Escalation wins over blanket processing.** Using the quantum classifier for all inputs resulted in lower accuracy than using the confidence-based escalation method, since the inherent stochastics of quantum measurements negatively affected already accurately classified inputs.

(c) **Anomaly detection and supervised classification play different roles.** These processes are not mutually exclusive but rather are complimentary. The 11 zero-days that were flagged through the anomaly gate detector would not have even been identified using either classifier.

XI. Future Work

A number of avenues for building upon this work have been identified, from short-term engineering solutions to long-term research initiatives.

A. Quantum Hardware Implementation

The first and most pressing task is applying this theoretical framework to physical quantum hardware. The IBM Quantum processor lineup (now featuring the 127-qubit Eagle chip accessible via the cloud) and IonQ's trapped-ion quantum computer offer attractive possibilities. It would be interesting to gauge the effect of noise in the system on the accuracy loss, and whether quantum error mitigation methods like zero-noise extrapolation and probabilistic error cancellation can maintain the classification accuracy found in the simulations.

Our initial expectation is positive. The circuit depth is fairly low (only 3 layers) and the number of qubits not very high (8 qubits). Nevertheless, the only realistic response is that time will tell, and experiments are currently underway.

B. Scaling Quantum Circuit Architecture

A larger number of qubits, beyond 8, would mean that the quantum classifier can handle higher-dimensional feature representations without resorting to PCA. Investigating other types of ansatze, such as hardware-efficient ansatze that leverage the unique quantum computer topology, may also result in additional gains in classification accuracy. Another approach to consider is quantum convolutional neural networks for sequential processing of HTTP data streams.

C. Assessment of Adversarial Robustness

An analysis using adversarial attack generation tools is necessary to measure the resilience of our quantum classifier in the face of attacks aimed at evading detection. Considering the mathematical properties of quantum feature spaces, it should be noted that the use of non-differentiable functions during quantum measurement makes it harder to calculate gradients needed by the adversary. This theoretical benefit needs to be empirically evaluated, which is why we will implement HTTP-specific adversarial attack generators.

D. Multi-Class Classification

In addition to the binary approach (Normal and Malicious classes), multi-class classification that specifies a particular attack type (SQLi, XSS, CMDi attacks etc.) instead of labelling the attack with a "malicious" tag could have been implemented. Having information that a request most likely includes SQL injections would have been much more helpful in analysing real cases than just being aware of its potential harm.

E. Evaluation on Real World Traffic Data

The usage of real world HTTP access logs from some widely used benchmarks such as CSIC 2010, ISCX, WAF-Brain, for training and validation of the classical and quantum models, would ensure their generalisability beyond the synthetic dataset used at the moment. Although synthetic data allows better control over classes' balance and variety of attacks, real traffic data poses many difficulties (such as malformed request, special encoding, non-English characters) not possible in the controlled environment.

F. Federated and Privacy-preserving Learning

Usage of federated learning methods for training the proposed models on multiple datasets without direct exchange of traffic data between organisations would overcome the main problem related to data protection. Federated learning would be especially helpful when working with the quantum neural network due to data insufficiency.

G. Learning Pipeline

To enable continuous learning by equipping the models with an ability to learn and adapt to new attack patterns without complete retraining will make the system more viable in the long run. The batch training method used currently freezes the knowledge of the system during training. Incorporating an active learning process wherein human annotators provide labels for anomalies in the data set will ensure that the system adapts to the changing threat environment.

H. Noise Simulation

The inclusion of depolarizing and amplitude damping noise in the simulations using PennyLane before evaluating the impact on QPUs will enable better predictions about how the attributes of NISQ-era hardware will influence escalation quality. This step can be considered an intermediary stage between noise-free simulations and hardware evaluation.

XII. Ethical Considerations

It is important to note that any research involving attack payload generation and analysis has ethical implications that should be addressed. The attack payload synthesis phase of the proposed framework creates realistic attack vectors in the form of malicious HTTP requests such as SQL injections, cross-site scripting payloads, and command injections. To mitigate potential misuse of these attack vectors, certain measures have been put in place.

Firstly, the attack payload generation phase does not create new attacks but synthetic copies of documented vulnerability exploits that have been listed in public documentation, such as the OWASP Testing Guide and the SecLists project. Secondly, the training data set is not being made publicly available but the detection framework and trained models. Thirdly, the proposed framework will only be used for defensive purposes within an organization's internal testing infrastructure by passing traffic through a local proxy.

From a privacy point of view, the framework parses the HTTP request body, which can include sensitive data, authentication credentials, or confidential business information. For a production setup, it is recommended that the pipeline be deployed within

an organization's data handling and privacy policy guidelines. The framework does not store or forward any information from the request body outside the classification pipeline, meaning all processing is ephemeral.

Lastly, we note that the confidence score information can be abused for malicious purposes as well. Specifically, an attacker might leverage the output confidence scores to continually tune the payload until low-confidence scores are returned. From a production standpoint, consideration should be given as to whether confidence scoring information should be exposed to clients or kept to internal backend logs only.

A. Bias and Fairness Considerations

Security algorithms based on ML face the problem of bias. If the algorithm was trained on a set of legitimate traffic which contains a disproportionate amount of legitimate traffic of specific kinds (for example, websites written in English with the Latin character set), it will exhibit false positives when evaluating traffic from other languages, encodings, and URLs. The 30-dimensional space we use is supposed to be neutral to languages since it works with statistics rather than the semantics of the message, but we lack enough evidence to make strong claims.

Another aspect that should not be overlooked is that of adversarial fairness. In case of successful adoption, the decision-making process of the algorithm will inevitably enter the realm of adversarial attacks. This leads to an arms race between the attacker and the defender, and while we assume that the quantum component makes our algorithm resilient to such attacks, recognizing the existence of an arms race should serve as a guideline in deployment decisions.

B. Responsible Disclosure

The payload creation module of this framework has been developed exclusively for defensive purposes. The produced payloads are based entirely on documented vulnerabilities. None of the exploits that have been developed as part of this research were novel. There are warnings included in the source code regarding the defensive nature of this project. We urge organizations implementing this framework to develop internal procedures to ensure compliance with defensive cybersecurity.

ACKNOWLEDGMENT

Acknowledgements

The authors thank the research teams of cybersecurity and quantum computing labs that have made possible the datasets used in this study through their contributions. The PennyLane library development team from Xanadu and the Burp Suite community are acknowledged for their documentation on extendability, while the dataset maintainers are acknowledged for their CSIC 2010 and SecLists datasets.

Computational resources were made available via institutional resources. No funding sources are declared by the authors.

REFERENCES

[1] A. et al., "Quantum AI for Cybersecurity: Hybrid Quantum-Classical Models for Attack Path Analysis," *arXiv preprint*, arXiv:2601.02237, 2026.
[2] B. et al., "Hybrid Quantum-Classical Autoencoders for Unsupervised Network Intrusion Detection," *arXiv preprint*, arXiv:2512.05069, 2025.
[3] C. et al., "Quantum AI Algorithm Development for Enhanced Cybersecurity: A Hybrid Approach to Malware Detection," *arXiv*, arXiv:2509.05370, 2025.
[4] D. Scholar, "Quantum Machine Learning Optimizations for Vulnerability Detection Sequences," *EPJ Quantum Technology*, doi:10.1140/epjqt/s40507-024-xxxx, 2024.
[5] E. Author, "Research Article on Hybrid Detection Matrices," *MDPI Electronics*, vol. 14, no. 9, art. 1827, 2024.
[6] F. Name et al., "QML-IDS: Quantum Machine Learning Intrusion Detection System," *IEEE Conf. Publication*, 2024.
[7] G. Researcher, "Survey on Vulnerability Detection in Web Environment Using Tools," *IEEE Access*, 2023.
[8] H. Science, "Developing Quantum-Enhanced Intelligent Systems for Ethical Cybersecurity," *IEEE Trans. Quantum Eng.*, 2023.
[9] I. Academia, "A Hybrid Framework Integrating QML, AI, and Quantum-Safe Cryptography," *J. Cybersec. Cryptogr.*, 2022.
[10] J. Quantum, "Quantum Machine Learning for Network Intrusion Detection System Validation," *IEEE Xplore*, 2022.
[11] K. Network et al., "Analysis of Parameter Shift Rules in Quantum Machine Learning," *EPJ Quantum Technology*, doi:10.1140/epjqt/s40507-023-00195-w, 2023.
[12] L. Cyber et al., "Discovery of Web Attacks by Inspecting HTTPS Network Traffic with ML," *Academic Dissertations*, 2024.
[13] M. Application, "A Review of Common Web Application Breaching Techniques," *Atlantis Press*, 2021.
[14] N. Suite, "SensitiveDiscoverer: A Burp Suite Extension for Automated HTTP String Proxy Scanning," *University Repositories*, 2020.
[15] O. Defense, "An In-Depth Comparative Study of Quantum-Classical Encoding Methods for IDS," *IEEE Xplore*, 2024.
[16] P. Tuning, "Fine-Tuned Variational Quantum Classifiers for Cyber Attacks Detection," *IEEE Conf. on Security*, 2023.
[17] Q. NLP, "HTTP2vec: Embedding of HTTP Requests for Anomalous Traffic Analysis," *arXiv preprint*, arXiv:2108.01763, 2021.
[18] R. Math, "Algorithmic Properties in Quantum Hilbert Spaces for Classification," *Math. Journals*, 2022.
[19] S. Grid, "Network Anomaly Detection Using Quantum Neural Networks on Noisy Hardware," *IEEE Xplore*, 2023.
[20] T. Detect, "Machine Learning for Detection and Mitigation of Web Vulnerabilities and Attacks," *arXiv*, arXiv:2304.14451, 2023.
[21] U. Security, "Implementation Validation of Python-Based Interception API Topologies," *Science Direct*, 2024.
[22] V. Optimization, "Hybrid-Quantum Neural Networks for Anomaly Detection and Optimization," *IEEE*, 2024.
[23] W. Cryptography, "Advances in Modern Cryptographic Techniques and QKD Models," *IEEE Xplore*, 2023.
[24] X. Request, "AlgoXSSF: Analysis of Cross-Site Request Forgery via Machine Learning," *IEEE*, 2023.
[25] Y. Intrusion, "Security Intrusion Detection Using Quantum Machine Learning Techniques," *Springer, J. Virus*, 2022.
[26] Z. Vulnerability, "VulDeeLocator: A Deep Learning-Based Fine-Grained Vulnerability Detector," *IEEE TDSC*, 2021.
[27] A.B. Security, "A Novel Anomaly Detection Pipeline Using Hybrid Systems," *Springer*, 2023.
[28] C.D. Scanners, "A Study of Vulnerability Scanners for Web Attack Mitigation," *AIA J.*, 2024.

[29] E.F. Automating, "Burp Suite: Automating Application Security Defenses," *ProQuest Collections*, 2020.

[30] G.H. Codebase, "Vudenc: Vulnerability Detection with Natural Codebases for Python," *ACM/IEEE ICSE*, 2023.

[31] A. Kadi, A. Selamnia, Z. A. E. Houda, H. Moudoud, B. Brik and L. Khoukhi, "An In-Depth Comparative Study of Quantum-Classical Encoding Methods for Network Intrusion Detection," *IEEE Open Journal of the Communications Society*, vol. 6, pp. 1129–1148, 2025.

[32] E. I. Elsedimy, H. Elhadidy and S. M. M. Abohashish, "A Novel Intrusion Detection System Based on a Hybrid Quantum Support Vector Machine and Improved Grey Wolf Optimizer," *Cluster Computing*, vol. 27, pp. 9917–9935, 2024.

[33] M. A. Rahman *et al.*, "Fine-Tuned Variational Quantum Classifiers for Cyber Attacks Detection Based on Parameterized Quantum Circuits and Optimizers," in *Proc. IEEE COMPSAC*, 2024, pp. 1067–1072.

[34] A. Kukliansky, M. Orescanin, C. Bollmann and T. Huffmire, "Network Anomaly Detection Using Quantum Neural Networks on Noisy Quantum Computers," *IEEE Transactions on Quantum Engineering*, vol. 5, pp. 1–11, 2024.

[35] I. Kahvazadeh, A. C. Fong and G. Hong, "A Framework for Hybrid-Quantum Neural Networks for Anomaly Detection and Optimization in Smart Grids," in *Proc. IEEE APPEEC*, 2025, pp. 1–5.

[36] M. Sajid, K. R. Malik, A. Almogren *et al.*, "Enhancing Intrusion Detection: A Hybrid Machine and Deep Learning Approach," *Journal of Cloud Computing*, vol. 13, 2024.

[37] M. Alghawazi, D. Alghazzawi and S. Alarifi, "Detection of SQL Injection Attack Using Machine Learning Techniques: A Systematic Literature Review," *Journal of Cybersecurity and Privacy*, vol. 2, no. 4, pp. 764–777, 2022.

[38] M. Gniewkowski, H. Maciejewski, T. R. Surmacz and W. Walentynowicz, "HTTP2vec: Embedding of HTTP Requests for Detection of Anomalous Traffic," *arXiv preprint*, arXiv:2108.01763, 2021.

[39] A. Mehmood, A. Shafique, M. Alawida and A. N. Khan, "Advances and Vulnerabilities in Modern Cryptographic Techniques: A Comprehensive Survey," *IEEE Access*, vol. 12, pp. 27530–27555, 2024.

[40] Y. Zheng, Z. Li, X. Xu and Q. Zhao, "Dynamic Defenses in Cyber Security: Techniques, Methods and Challenges," *Digital Communications and Networks*, vol. 8, no. 4, pp. 422–435, 2022.